TECHNICAL REPORT · WAVEFORMRS

Validating, optimising and extending waveformrs against LALSimulation

The crate as it was found, the bugs that separated it from LALSimulation, the tests that now hold every model to LALSimulation 6.2.0 at rounding level, the optimisations (including for generating many waveforms in sequence), and new ports of IMRPhenomXAS, XHM, XP and XPHM with SpinTaylor, MSA and NNLO precession angles, IMRPhenomXPNR, and of the NR surrogate NRSur7dq4, with a common interface, batching, domain conversions and Linux testing.

BRANCH
`xphm-spintaylor-validation`, 148 commits on `main` (`87f9d9b`)

REFERENCE
lalsimulation 6.2.0, lal 7.7.0, GSL 2.8 (conda-forge, osx-arm64)

PLATFORM
Apple M2 Ultra, macOS 26.6, rustc 1.86.0-nightly

GENERATED
18 October 2026

CONTENTS

1 Summary
2 The crate as found
3 Defects found and fixed
4 How the models are tested
5 Accuracy
6 Performance
7 Many waveforms in sequence
8 Long, low-mass waveforms
9 New models
10 IMRPhenomXPHM with SpinTaylor angles
11 NRSur7dq4
12 IMRPhenomXPNR
13 Using the crate
14 Python bindings
15 Other platforms
16 Issues found in LALSimulation
17 API review and suggested changes
18 Limitations and future work
19 Reproducing the results
20 Commit history

1 Summary

The crate started with two models, TaylorF2 and IMRPhenomD. It had no tests, and both models disagreed with LALSimulation by mismatches of up to 0.14. Both now agree with LALSimulation at rounding level and run about twice as fast as before. Eight more model configurations were ported, ending with IMRPhenomXPHM using the numerical SpinTaylor precession angles, followed by the time-domain NR surrogate NRSur7dq4 and IMRPhenomXPNR. XPNR includes, by default, a fix from lalsuite master that restores its mirror symmetry. All models share one interface with LAL's conventions, which is also available from Python. They can be batched over threads, convert between the time and frequency domains, and have been tested on Linux as well as macOS. Every model is checked against LALSimulation by `cargo test` and by 2,600 random parameter draws. For samplers, which evaluate many waveforms on one frequency grid, a grid path removes the remaining per-frequency transcendental functions.

AGREEMENT WITH LALSIMULATION

$\leq 4.6\times10^{-11}$

Largest relative error, $|h - h_{LAL}| / \max |h_{LAL}|$, over all 2,600 random draws. The largest mismatch is 4.4×10^{-16}.

REFERENCE CASES IN `CARGO TEST`

395

In 44 files generated with lalsimulation 6.2.0, checked to ≤ 1e-9 relative error.

IMRPHENOMXPHM, SPINTAYLOR ANGLES

3.35 ms vs 8.58 ms

Rust against LALSimulation on one thread, for 5921 frequencies (2.6× faster). On a uniform grid with LAL's default multibanding, LAL takes 6.22 ms and Rust 3.38 ms.

NRSUR7DQ4

bit-identical, 5.7×

Faster than LALSimulation for the polarizations; 0.03–0.15 ms when only extrinsic parameters change. The surrogate data is read by a 912-line pure-Rust HDF5 reader.

MANY WAVEFORMS, ONE GRID

2.3×–7.8×

Speed-up over LALSimulation per waveform when 300 random binaries are evaluated in sequence on one 8033-point grid (section 7).

ORIGINAL MODELS

2.1× / 1.9×

Speed-up of TaylorF2 and IMRPhenomD over the original `main`, on the same 5921-point grid.

MODEL	STATUS	DRAWS	MAX REL. ERROR	MAX MISMATCH	RUST	LAL	SPEED-UP
TaylorF2	fixed	200/200	2.2×10^{-11}	3.3×10^{-16}	110 µs	172 µs	1.6×
IMRPhenomD	fixed	200/200	2.2×10^{-11}	4.4×10^{-16}	142 µs	306 µs	2.2×
IMRPhenomXAS	new	200/200	4.6×10^{-11}	3.3×10^{-16}	103 µs	208 µs	2.0×
IMRPhenomXHM	new	200/200	2.5×10^{-15}	3.3×10^{-16}	1.06 ms	1.74 ms	1.6×
IMRPhenomXP, NNLO (102)	new	200/200	1.6×10^{-15}	4.4×10^{-16}	347 µs	654 µs	1.9×
IMRPhenomXP, MSA (223)	new	200/200	2.2×10^{-15}	3.3×10^{-16}	1.13 ms	1.55 ms	1.4×
IMRPhenomXP, SpinTaylor (320)	new	200/200	1.6×10^{-15}	3.3×10^{-16}	765 µs	2.86 ms	3.7×
IMRPhenomXPHM, NNLO (102)	new	200/200	1.5×10^{-15}	3.3×10^{-16}	2.03 ms	4.40 ms	2.2×
IMRPhenomXPHM, MSA (223)	new	200/200	2.1×10^{-15}	2.2×10^{-16}	5.07 ms	8.81 ms	1.7×
IMRPhenomXPHM, SpinTaylor (320)	new	200/200	1.4×10^{-15}	3.3×10^{-16}	3.35 ms	8.58 ms	2.6×
IMRPhenomXPNR (6.2.0, no parity fix)	new	200/200	3.2×10^{-15}	4.4×10^{-16}	3.86 ms	9.41 ms	2.4×
IMRPhenomXPNR with the parity fix (patched LAL)	new	200/200	3.2×10^{-15}	2.2×10^{-16}	3.86 ms	9.41 ms	2.4×
NRSur7dq4 (time domain)	new	200/200	0	0	1.01 ms	5.78 ms	5.7×

Accuracy from 200 random draws per model. Timings are for construction plus polarizations on 5921 frequencies (20–1500 Hz, $\Delta f = 0.25$ Hz), single thread, against `SimInspiralChooseFDWaveformSequence` with multibanding off. The aligned-spin models use $M = 60\,M_\odot$, $q = 0.5$, $\chi = (0.3, -0.2)$. The precessing models use $m_1 = 40$, $m_2 = 20\,M_\odot$, $\chi_1 = (0.4, -0.3, 0.2)$, $\chi_2 = (0.1, 0.5, -0.3)$.

The four tasks and their outcomes:

- **Assessment.** Section 2 describes the crate as found. The defects that separated it from LALSimulation are listed in section 3.
- **Testing against LALSimulation.** Every model, including each precession-angle option and each fallback path, has reference data from LALSimulation in `tests/data`. A harness compares the models over random draws (section 4, section 5).
- **Optimisation.** Every model runs faster than LALSimulation on one thread, without changing the existing API (section 6). Section 17 lists API changes that would give more.
- **IMRPhenomXPHM with SpinTaylor angles.** This is implemented, together with the models and integrator it builds on (section 9, section 10). The SpinTaylor Euler angles agree with LALSimulation's bit for bit, and the polarizations agree to ≤ 1e-15.

> **LALSimulation bug found along the way.** In multi-threaded builds, IMRPhenomXP's default MSA angles and its PNR angles are wrong at some frequencies, and the output changes from run to run. Errors reach several percent of the peak amplitude, and are of order one for PNR. The cause is a data race on `IMRPhenomXPrecessionStruct` inside an OpenMP loop. Section 16 summarises it. The full write-up with the lines to fix is `report/lalsimulation-openmp-race.md`.

2 The crate as found

On `main` (`87f9d9b`, a single commit) the crate had about 1,600 lines of Rust in 13 files:

- A **`Waveform` trait** with `orbital_speed`, `phase` and `amplitude`, plus default `waveform_single_frequency`, `waveform` and `waveform_modes` methods. The last one returns h_+ and $h_\times$ for an inclination.
- **TaylorF2**: post-Newtonian phasing up to 3.5PN with spin-orbit, spin-spin and tidal terms, and PN amplitude corrections (`src/pn.rs`, `src/taylor.rs`).
- **IMRPhenomD**: the phenomenological inspiral–merger–ringdown model, with its amplitude collocation system solved by LAPACK through `ndarray-linalg` (`src/imrphenomd/`).
- **Dependencies:** `ndarray`, `ndarray-linalg` and `blas-src` (Accelerate), with macOS-specific link flags in `.cargo/config.toml`. `autodiff` was listed but unused.
- **Tests:** none, and no reference data or comparison against LALSimulation. Two examples printed timings.

To measure where it stood, I compared the original code with LALSimulation on 100 random aligned-spin binaries per model. The match is maximised over time and phase, so differing phase conventions don't count against it. The mismatch is weighted by the aLIGO design sensitivity.

ORIGINAL MODEL	MEDIAN MISMATCH	90TH PERCENTILE	MAX MISMATCH	DRAWS WITH MISMATCH > 1E-3	MAX AMPLITUDE ERROR	TIME (5921 FREQUENCIES)
TaylorF2	5.7×10^{-4}	9.5×10^{-3}	1.4×10^{-1}	40 / 100	1.3×10^{-15}	235 µs
IMRPhenomD	4.1×10^{-4}	4.3×10^{-3}	9.7×10^{-3}	30 / 100	2.6×10^{-1}	274 µs

100 draws per model: $M \in [5, 200]\ M_\odot$ log-uniform, $q \in [1/20, 1]$ (1/18 for IMRPhenomD), $\chi_i \in [-0.9, 0.9]$, 20 Hz up to the model's cutoff, $\Delta f = 0.125$ Hz. Script: `report/baseline_accuracy.py`.

Mismatches of 1e-3 to 1e-1 are large enough to bias parameter estimation for loud signals, so the original crate couldn't stand in for LALSimulation.

3 Defects found and fixed

Comparing the coefficients one by one with LALSimulation turned up these defects. Each fix was checked against reference waveforms.

WHERE	DEFECT	EFFECT	FIX
TaylorF2 phase, `pn::taylor_3pn_ss`	The 3PN spin–spin term $\chi_1\chi_2$ was missing a factor of η.	Wrong phase for every binary with two spinning bodies. This drives the TaylorF2 mismatches of up to 0.14.	`62ff04a`
PN amplitude coefficients, `pn::taylor_f2_amplitude_*`	Used $q = m_2/m_1$ where $\delta = (m_1 - m_2)/M$ is needed.	Wrong amplitude corrections for unequal masses.	`62ff04a`
IMRPhenomD inspiral amplitude	Used the generic PN amplitude series instead of the prefactors IMRPhenomD was calibrated with (`init_amp_ins_prefactors`), whose 2.5PN and 3PN terms differ.	Inspiral amplitude errors, and through the collocation system, errors in the intermediate amplitude.	`7cfa480`
IMRPhenomD ringdown and damping frequencies	Used rational fits in the final spin. LALSimulation interpolates tabulated quasi-normal-mode data with a natural cubic spline and divides by $1 - E_{rad}$.	Shifted merger–ringdown, with amplitude errors of up to 26%.	`7cfa480`: tabulated QNM data and a port of `gsl_interp_cspline`
Reference phase and frequency	There was no way to set the reference frequency, so outputs couldn't be lined up with LALSimulation's `f_ref` / `phi_ref`.	Phase and time conventions differed from LAL.	New builder `with_reference_frequency` on both models, with existing behaviour unchanged by default.

The fixes remove all of the measured discrepancy. After them both models agree with LALSimulation to ~1e-11 (section 5). The LAPACK dependency was replaced by a 5×5 Gaussian elimination, which removes the platform-specific BLAS linking.

4 How the models are tested

Reference data in `cargo test`

`scripts/generate_reference_data.py` calls lalsimulation and stores waveforms as JSON in `tests/data`; `serde_json` is built with `float_roundtrip` so no bits are lost. The waveforms come from `SimInspiralChooseFDWaveformSequence` on non-uniform grids and `SimInspiralChooseFDWaveform` on uniform grids, for both polarizations. Individual higher modes come from `SimIMRPhenomXHMFrequencySequenceOneMode`, Euler angles from `SimIMRPhenomXPSpinTaylorAngles`, and orbits from `SimInspiralSpinTaylorT4`. The cases are chosen to cover:

- equal and extreme mass ratios, down to 1/120 for the higher modes;
- spins up to 0.95;
- a $1.6 + 1.3\ M_\odot$ binary, whose inspiral phase reaches ~10^6 rad;
- heavy binaries that trigger LAL's fallbacks;

- non-default reference frequencies, uniform grids and restricted mode arrays.

The integration tests in `tests/lalsimulation.rs` and `tests/spintaylor.rs` require a relative error ≤ 1e-10 (TaylorF2, IMRPhenomD) or ≤ 1e-9 (IMRPhenomX family), and a mismatch ≤ 1e-14 or ≤ 1e-12. In practice the errors are far below these limits:

TEST	COMPARISONS	WORST REL. ERROR	WORST MISMATCH
TaylorF2 polarizations `taylorf2_matches_lalsimulation`	24	2.2×10^{-12}	3.3×10^{-16}
IMRPhenomD polarizations `imrphenomd_matches_lalsimulation`	16	7.2×10^{-12}	5.6×10^{-16}
IMRPhenomXAS polarizations `imrphenomxas_matches_lalsimulation`	16	1.4×10^{-11}	3.3×10^{-16}
IMRPhenomXHM polarizations `imrphenomxhm_matches_lalsimulation`	16	2.9×10^{-16}	1.1×10^{-16}
IMRPhenomXHM individual modes (both signs of m) `imrphenomxhm_modes_match_lalsimulation`	43	2.7×10^{-16}	1.1×10^{-16}
IMRPhenomXP, NNLO angles `imrphenomxp_nnlo_matches_lalsimulation`	16	5.3×10^{-16}	1.1×10^{-16}
IMRPhenomXP, MSA angles (incl. MSA → NNLO fallback) `imrphenomxp_msa_matches_lalsimulation`	22	4.2×10^{-16}	1.1×10^{-16}
IMRPhenomXP, SpinTaylor angles `imrphenomxp_spintaylor_matches_lalsimulation`	16	5.0×10^{-16}	4.4×10^{-16}
IMRPhenomXP, SpinTaylor → MSA fallback `imrphenomxp_spintaylor_fallback_matches_lalsimulation`	16	8.5×10^{-16}	3.3×10^{-16}
IMRPhenomXP, uniform grids `imrphenomxp_uniform_matches_lalsimulation`	8	3.8×10^{-16}	1.1×10^{-16}
SpinTaylor Euler angles (SimIMRPhenomXPSpinTaylorAngles) `imrphenomxp_spintaylor_angles_match_lalsimulation`	4	0 bit-identical	–
IMRPhenomXPHM, SpinTaylor angles `imrphenomxphm_spintaylor_matches_lalsimulation`	16	4.2×10^{-16}	1.1×10^{-16}
IMRPhenomXPHM, NNLO angles `imrphenomxphm_nnlo_matches_lalsimulation`	16	4.9×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, MSA angles `imrphenomxphm_msa_matches_lalsimulation`	22	3.9×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, SpinTaylor → MSA fallback `imrphenomxphm_spintaylor_fallback_matches_lalsimulation`	16	6.4×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, uniform grids `imrphenomxphm_uniform_matches_lalsimulation`	10	4.2×10^{-16}	2.2×10^{-16}
IMRPhenomXPHM, restricted mode arrays `imrphenomxphm_mode_arrays_match_lalsimulation`	12	3.9×10^{-16}	≤ 0
SpinTaylorT4 orbit evolutions `spintaylor_t4_matches_lalsimulation`	15	0 bit-identical	–
NRSur7dq4 polarizations (ChooseTDWaveform) `nrsur7dq4_polarizations_match_lalsimulation`	19	0 bit-identical	–
NRSur7dq4 modes (ChooseTDModes) `nrsur7dq4_modes_match_lalsimulation`	96	0 bit-identical	–
NRSur7dq4 dynamics `nrsur7dq4_dynamics_match_lalsimulation`	4	0 bit-identical	–
HDF5 reader against h5py (datasets) `hdf5_reader_matches_h5py`	4776	0 bit-identical	–
IMRPhenomXHM release 122019 polarizations `imrphenomxhm_122019_matches_lalsimulation`	16	4.0×10^{-16}	2.2×10^{-16}
IMRPhenomXHM release 122019 modes `imrphenomxhm_122019_modes_match_lalsimulation`	58	0 bit-identical	≤ 0
IMRPhenomXPNR polarizations (incl. every regime switch) `imrphenomxpnr_matches_lalsimulation`	64	1.3×10^{-15}	2.2×10^{-16}
IMRPhenomXPNR with the parity fix (6.2.0 + lalsuite !2593) `imrphenomxpnr_parity_fix_matches_patched_lalsimulation`	64	1.2×10^{-15}	2.2×10^{-16}
IMRPhenomXPNR, uniform grids (with and without the fix) `imrphenomxpnr_uniform_matches_lalsimulation`	28	6.9×10^{-16}	2.2×10^{-16}
IMRPhenomXPNR modes (ChooseFDModes) `imrphenomxpnr_choose_fd_modes_matches_lalsimulation`	234	3.9×10^{-16}	2.2×10^{-16}

TEST	COMPARISONS	WORST REL. ERROR	WORST MISMATCH
IMRPhenomXPNR modes on frequency sequences `imrphenomxpnr_sequence_modes_match_lalsimulation`	108	4.0×10^{-16}	≤ 0
Modes of IMRPhenomXHM/XPHM (ChooseFDModes) `choose_fd_modes_matches_lalsimulation`	167	3.6×10^{-16}	4.4×10^{-16}
Modes on frequency sequences `modes_on_frequency_sequences_match_lalsimulation`	71	3.6×10^{-16}	1.1×10^{-16}
Common interface against the reference suites `choose_fd_model_matches_lalsimulation`	192	1.4×10^{-11}	5.6×10^{-16}
SimInspiralFD conditioning of the FD models `conditioned_fd_matches_lalsimulation`	12	4.0×10^{-13}	–
SimInspiralFD, NRSur7dq4 `nrsur7dq4_fd_matches_lalsimulation`	16	4.1×10^{-16}	–
SimInspiralTD, NRSur7dq4 `nrsur7dq4_td_matches_lalsimulation`	4	0 `bit-identical`	–
SimInspiralTD, TaylorF2 `taylorf2_td_matches_lalsimulation`	6	3.6×10^{-11}	–
SimInspiralTD, IMRPhenomD `imrphenomd_td_matches_lalsimulation`	10	3.1×10^{-12}	–
SimInspiralTD, IMRPhenomXAS `imrphenomxas_td_matches_lalsimulation`	10	2.5×10^{-12}	–
SimInspiralTD, IMRPhenomXHM `imrphenomxhm_td_matches_lalsimulation`	10	2.3×10^{-12}	–
SimInspiralTD, IMRPhenomXP `imrphenomxp_td_matches_lalsimulation`	4	1.8×10^{-12}	–
SimInspiralTD, IMRPhenomXPHM `imrphenomxphm_td_matches_lalsimulation`	8	2.1×10^{-12}	–
SimInspiralTD, IMRPhenomXPNR `imrphenomxpnr_td_matches_lalsimulation`	6	1.6×10^{-12}	–

Output of `cargo test --release --test lalsimulation -- --nocapture`. Each comparison is one polarization or mode of one case. A mismatch below zero means the two agree to rounding.

Random draws

`scripts/validate_against_lalsimulation.py` draws 200 binaries per model and evaluates both codes on the same uniform grid. Rust is driven through `examples/lal_validation.rs`, which reads JSON on stdin.

- **Parameters:** M log-uniform in [5, 300] $M_\odot$; q uniform in [1/20, 1] (1/18 for IMRPhenomD); aligned spins uniform in ±0.99, or isotropic spin vectors with $|\chi| \leq 0.99$ for the precessing models.
- **Frequencies:** $f_{min} \in \{10, 15, 20\}$ Hz; $f_{ref} = f_{min}$, or uniform in $[f_{min}, 3\, f_{min}]$; $\Delta f = 0.25$ Hz up to the model's cutoff or 2048 Hz.
- **Extrinsic:** isotropic inclination; uniform reference phase; distance log-uniform in [100, 3000] Mpc.

Each draw records three things:

- the largest relative error;
- the noise-weighted mismatch (aLIGO zero-detuned high-power PSD), with *no* maximisation over time or phase, so a constant phase offset would show up;
- the median time of five calls for each code.

When LALSimulation silently changes method, the harness records the case separately and compares it against the method LAL fell back to, as in the SpinTaylor → MSA case (section 16). In the final run every draw was compared directly.

Agreement to the last bit

Matching to ~1e-9 needs the same formulas. For three components, matching better than that needs the same floating-point operations:

- **The SpinTaylor integrator.** Its adaptive step-size control turns last-bit differences in the derivatives into different accepted steps, and those grow to ~1e-7 in the spins.
- **The IMRPhenomXHM (3,2) mode.** Its intermediate phase uses a finite-difference second derivative with a step of 1e-7, which amplifies rounding by ~10^{14}. Even LALSimulation's own (3,2) mode changes by up to 3e-3 when the inputs move by 1e-15.

- **The cubic splines and final-state fits** that set the ringdown frequencies, and through them the SpinTaylor time step.

On Apple silicon, clang compiles LALSimulation and GSL with `-ffp-contract=on`. It fuses `a*b + c` into a single-rounding fused multiply-add wherever the product and the sum appear in the same expression, and OpenBLAS (behind GSL's LU solves) fuses as well. `src/spintaylor/fp.rs` provides `fma`, `fms` and `fnms` helpers that reproduce clang's contraction rule, and the ported code uses them operation by operation. The parameter-space fits of IMRPhenomXAS and XHM were generated from the C sources with the same rule. With this the SpinTaylor orbit, the splines, the remnant fits, every IMRPhenomX/XHM coefficient and the SpinTaylor and MSA Euler angles are bit-identical to LALSimulation on this platform.

On x86-64, builds without hardware FMA don't contract, and the helpers fall back to separate rounding. That configuration hasn't been tested (section 18).

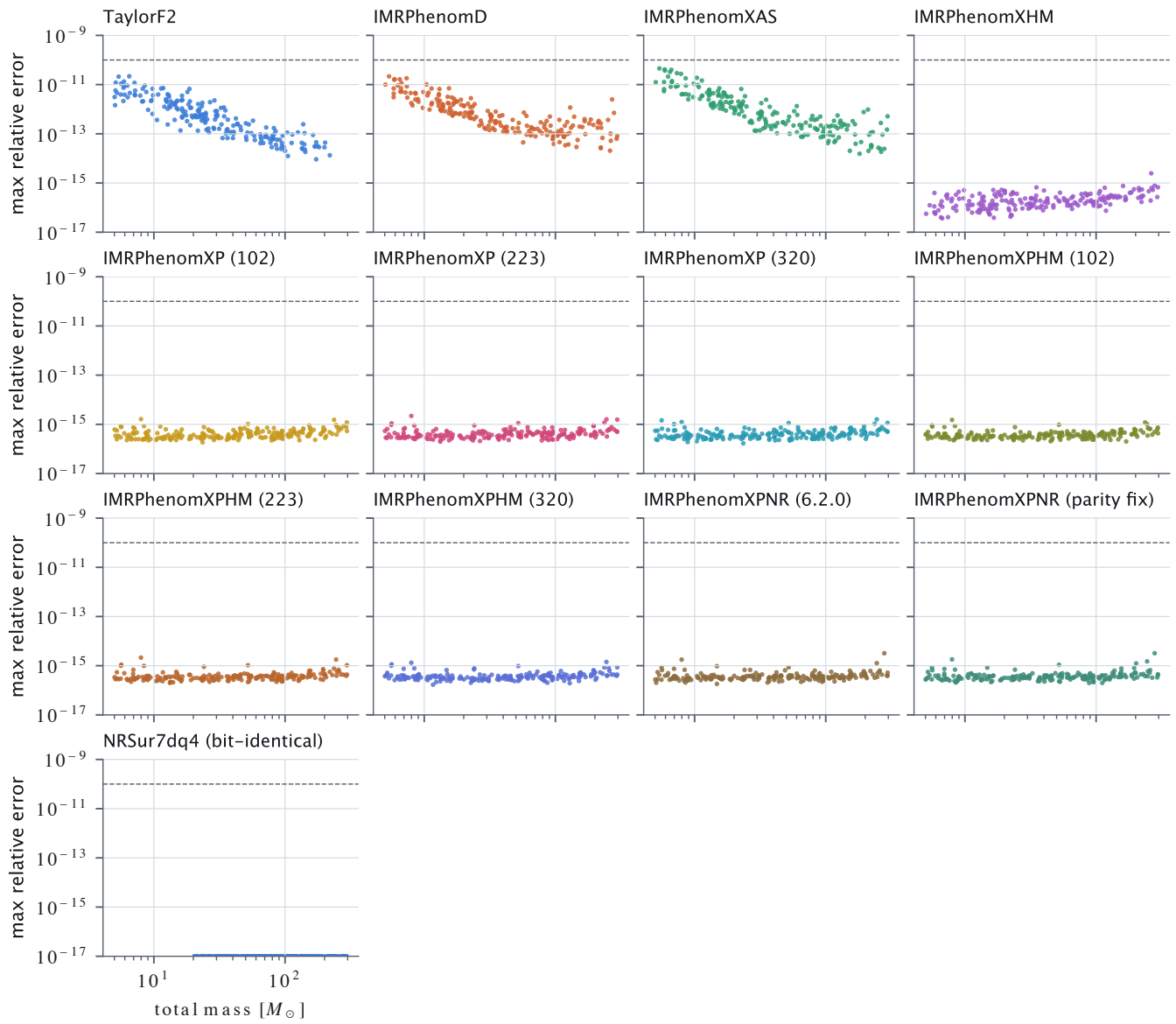
Threads

All LALSimulation runs use `OMP_NUM_THREADS=1`. Besides giving fair single-core timings, this is needed because multi-threaded IMRPhenomXP is not deterministic with the MSA angles (section 16).

5 Accuracy

MODEL	DRAWS	MAX REL. ERROR	MEDIAN REL. ERROR	MAX MISMATCH	MEDIAN MISMATCH
TaylorF2	200/200	2.2×10^{-11}	4.4×10^{-13}	3.3×10^{-16}	0
IMRPhenomD	200/200	2.2×10^{-11}	4.9×10^{-13}	4.4×10^{-16}	0
IMRPhenomXAS	200/200	4.6×10^{-11}	6.5×10^{-13}	3.3×10^{-16}	0
IMRPhenomXHM	200/200	2.5×10^{-15}	1.8×10^{-16}	3.3×10^{-16}	0
IMRPhenomXP, NNLO (102)	200/200	1.6×10^{-15}	3.7×10^{-16}	4.4×10^{-16}	0
IMRPhenomXP, MSA (223)	200/200	2.2×10^{-15}	3.8×10^{-16}	3.3×10^{-16}	0
IMRPhenomXP, SpinTaylor (320)	200/200	1.6×10^{-15}	3.7×10^{-16}	3.3×10^{-16}	0
IMRPhenomXPHM, NNLO (102)	200/200	1.5×10^{-15}	3.6×10^{-16}	3.3×10^{-16}	0
IMRPhenomXPHM, MSA (223)	200/200	2.1×10^{-15}	3.5×10^{-16}	2.2×10^{-16}	0
IMRPhenomXPHM, SpinTaylor (320)	200/200	1.4×10^{-15}	3.5×10^{-16}	3.3×10^{-16}	0
IMRPhenomXPNR (6.2.0, no parity fix)	200/200	3.2×10^{-15}	3.5×10^{-16}	4.4×10^{-16}	0
IMRPhenomXPNR with the parity fix (patched LAL)	200/200	3.2×10^{-15}	3.4×10^{-16}	2.2×10^{-16}	0
NRSur7dq4 (time domain)	200/200	0	0	0	0

200 random draws per model, against lalsimulation 6.2.0 with `OMP_NUM_THREADS=1`. Mismatches are not maximised over time or phase. Data: `report/data/validation_*.json`.



Largest relative error of each random draw against total mass. The dashed line is 1e-10. The ~1e-11 tail of TaylorF2, IMRPhenomD and IMRPhenomXAS comes from low-mass binaries. Their inspiral phase reaches 10^5–10^6 rad, where one unit in the last place is 1e-11 to 1e-10 rad, so these models agree with LAL to within a few roundings of the phase. The higher-mode and precessing models evaluate the (2,2) phase with LALSimulation's exact powers of Mf and operation order, so they agree to ~1e-15.

No draw comes near the tolerances of the integration tests, and every mismatch is at the level of double-precision rounding ($\leq 4.4\times10^{-16}$). There is a trade-off for the (2,2)-only models. IMRPhenomXAS evaluates its powers of Mf with one `cbrt`, one `ln` and multiplications rather than LAL's sequence of `pow` calls, which buys about 20% in speed for errors at the 1e-11 level.

6 Performance

What was changed

- **One set of transcendental calls per frequency.** TaylorF2 and IMRPhenomD now compute the shared powers of v or Mf once per frequency: one `cbrt`, one `ln` and one `sin_cos`. Amplitude and phase both use them, and nothing is evaluated twice through the trait's separate `phase` / `amplitude` methods (`5f426f9`).

- **Polynomial evaluation.** The PN phase polynomials use Estrin's scheme instead of Horner's. TaylorF2 has a fast path for point-particle phasing and falls back to a sparse Horner evaluation when tidal terms are present (`d44fb0f`). `powf` and `powi` were removed from hot loops.
- **No LAPACK.** The IMRPhenomD amplitude collocation uses a fixed-size Gaussian elimination, and the IMRPhenomX collocation systems use a port of GSL's LU solve. The setup costs 0.4–1 µs per waveform.
- **Buffer reuse.** The default `waveform_modes` builds h_+ in place in the waveform buffer instead of allocating three vectors.
- **Precessing models.**
  - Euler angles and twisting factors ($e^{ik\alpha}$, powers of the half angles, the ε phase) are computed once per frequency and per m′, and shared by all modes with that m′.
  - No allocation happens per mode or per frequency.
  - The SpinTaylor γ integration stops 64 knots past the last frequency it is needed for, and repeated Boole-rule sums are cached. The output is bit-identical.
  - The degenerate MSA case skips the spin-magnitude cubic.
- **Whole-program optimisation.** Adding the IMRPhenomXHM modules changed LLVM's inlining decisions and slowed IMRPhenomXAS from 112 to 151 µs, even though the XAS code was unchanged. A single codegen unit doesn't recover it; fat LTO does, bringing it to ~105 µs. The release profile now sets `lto = "fat"` (`3719da8`). Crates that depend on waveformrs should do the same.

Timings

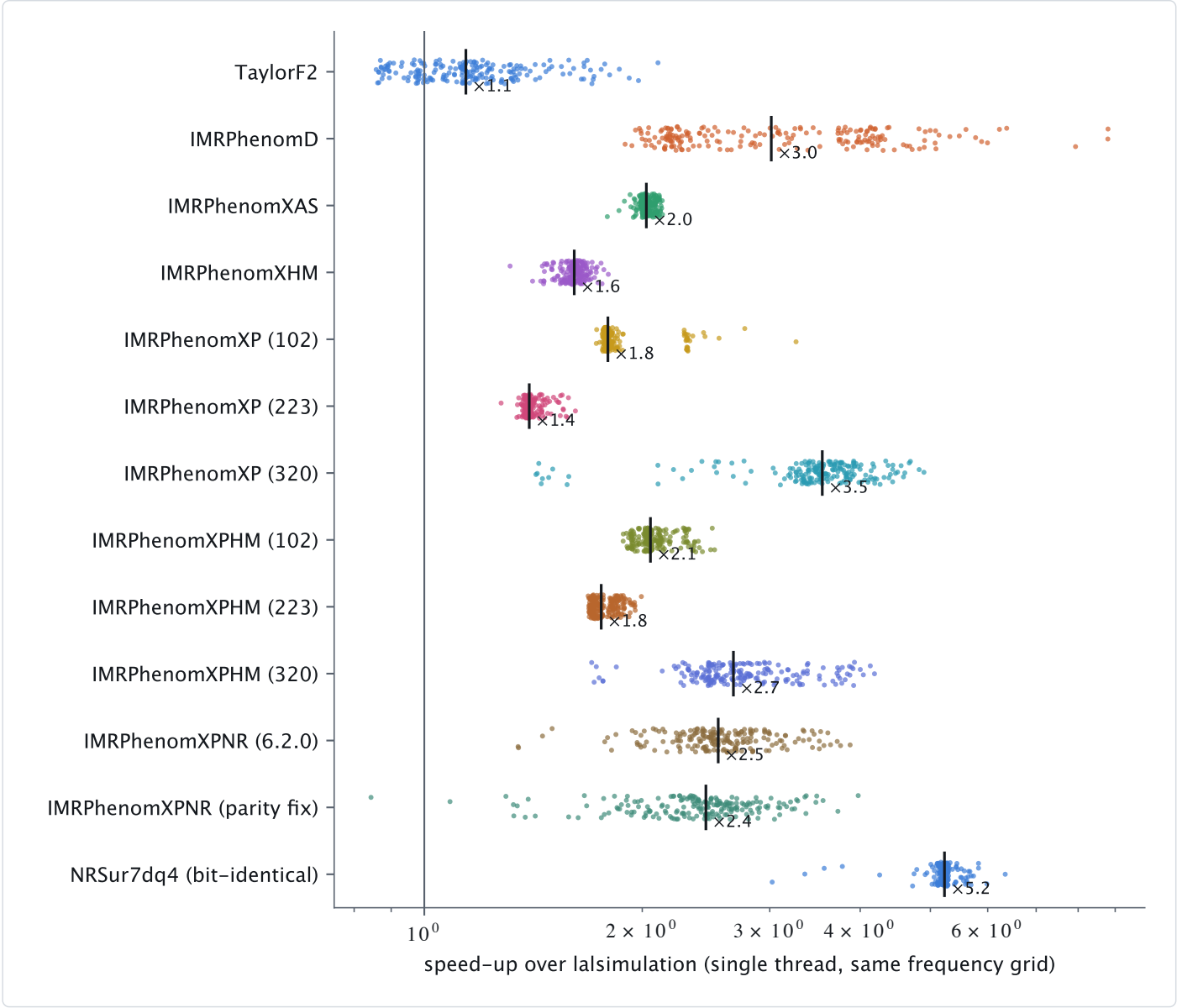
MODEL	ORIGINAL `MAIN`	RUST NOW	LAL	SPEED-UP
TaylorF2	235 µs	110 µs	172 µs	1.6×
IMRPhenomD	274 µs	142 µs	306 µs	2.2×
IMRPhenomXAS	–	103 µs	208 µs	2.0×
IMRPhenomXHM	–	1.06 ms	1.74 ms	1.6×
IMRPhenomXP, NNLO (102)	–	347 µs	654 µs	1.9×
IMRPhenomXP, MSA (223)	–	1.13 ms	1.55 ms	1.4×
IMRPhenomXP, SpinTaylor (320)	–	765 µs	2.86 ms	3.7×
IMRPhenomXPHM, NNLO (102)	–	2.03 ms	4.40 ms	2.2×
IMRPhenomXPHM, MSA (223)	–	5.07 ms	8.81 ms	1.7×
IMRPhenomXPHM, SpinTaylor (320)	–	3.35 ms	8.58 ms	2.6×
IMRPhenomXPNR (6.2.0, no parity fix)	–	3.86 ms	9.41 ms	2.4×
IMRPhenomXPNR with the parity fix (patched LAL)	–	3.86 ms	9.41 ms	2.4×
NRSur7dq4 (time domain)	–	1.01 ms	5.78 ms	5.7×

Best of 10 batches, construction plus polarizations, 5921 frequencies, single thread, against `SimInspiralChooseFDWaveformSequence` . Scripts: `examples/benchmark.rs` and `report/lal_benchmark.py` . Data: `report/data/timings.json` .

LALSimulation's higher-mode models can also *multiband* on uniform grids: they evaluate the modes on coarser grids and interpolate. This is on by default in `SimInspiralChooseFDWaveform` , which also stops at Mf = 0.3 (1015 Hz for this binary). Rust doesn't multiband. On the same uniform grid it is still faster than LAL with multibanding:

MODEL, UNIFORM GRID TO MF = 0.3	RUST	LAL, MULTIBANDING OFF	LAL, DEFAULT MULTIBANDING	SPEED-UP OVER LAL WITH MULTIBANDING
IMRPhenomXHM	717 µs	1.24 ms	1.05 ms	1.5×
IMRPhenomXPHM, SpinTaylor (320)	3.38 ms	8.41 ms	6.22 ms	1.8×

Δf = 0.25 Hz from 20 Hz, about 3,980 frequencies. IMRPhenomXHM is timed in Rust on the same frequencies with `polarizations` ; IMRPhenomXPHM with `polarizations_uniform` .



Speed-up over LALSimulation for each random draw (median of five calls each, the same grid, one thread). Bars mark the medians. TaylorF2 gains least on short grids, where fixed per-call costs on both sides dominate.

Where the time goes in the precessing models

For IMRPhenomXPHM with SpinTaylor angles on the benchmark grid (about 4 ms):

- the PN integration with the ported RKF45 integrator takes about 0.6 ms;
- the γ grid (minimal-rotation integral) takes about 1.2 ms;
- the five co-precessing modes take about 0.9 ms;
- angles and twist factors at four values of m′ take about 1.0 ms;
- twisting the modes up takes about 0.3 ms.

The MSA angles cost about 15 transcendental functions per frequency. These include a Jacobi elliptic function, ported from GSL's `gsl_sf_elljac_e` with its descending Landen transformation, and they are evaluated once per m′. MSA is therefore the slowest precessing variant in Rust, and the one with the smallest speed-up over LAL. All of these steps are bit-exact ports; changing their arithmetic would trade agreement with LAL for speed.

7 Many waveforms in sequence

A stochastic sampler evaluates one waveform after another, each with new parameters, on the same frequency grid. Two kinds of work can then be done once rather than for every waveform: the part of each transcendental function that depends only on the frequency, and the allocation of the output.

The grid path

Every frequency-domain model depends on the frequency only through a scaled frequency s·f, for example Mf or πMf. So its cube root, logarithm and inverse factorise: cbrt(s f) = cbrt(s)·cbrt(f) and ln(s f) = ln s + ln f.

- `FrequencyGrid::new(&frequencies)` stores $f^{1/3}$, $f^{1/6}$, 1/f, ln f and $f^{-7/6}$ once.
- Each waveform then computes those functions once for its own scale s, and only multiplies at each frequency.
- The new methods write into buffers the caller owns, so nothing is allocated per call: `Waveform::waveform_on_grid`, `Waveform::waveform_modes_on_grid`, `IMRPhenomXHM::polarizations_on_grid`, and the same for IMRPhenomXP and XPHM.

```
use waveformrs::{grid::FrequencyGrid, imrphenomx::IMRPhenomXAS, waveform::Waveform};

let grid = FrequencyGrid::uniform(20.0, 1024.0, 0.125);           // once
let mut hp = vec![Complex64::default(); grid.len()];
let mut hc = vec![Complex64::default(); grid.len()];
for p in &samples {                                               // many times
    IMRPhenomXAS::new(p.total_mass, p.mass_ratio, p.chi_1, p.chi_2, p.distance)
        .with_reference_frequency(20.0)
        .waveform_modes_on_grid(&grid, p.phi_ref, p.inclination, &mut hp, &mut hc);
    // ... likelihood ...
}
```

Once the powers come from the grid, `sin_cos` of the phase is the largest remaining cost, so the grid paths work in two passes:

1. compute every amplitude and phase;
2. convert them to complex values with an inlined `sin_cos`, which the compiler vectorises with NEON.

This `sin_cos` uses Cody–Waite reduction with fused multiply-adds and the fdlibm kernel polynomials, and stays within 3 ulp of the C library. A vectorised `exp`, within 3 ulp of libm, handles the SpinTaylor merger–ringdown angles and the γ integral. The grid paths agree with the per-frequency paths to ~1e-13 (`tests/grid.rs`). The per-frequency paths are unchanged, so they still match LALSimulation at rounding level.

Memory layout turned out to matter. When the five power arrays of a grid landed a whole number of pages apart, they competed for the same cache sets. The same benchmark binary then took 92 or 197 µs for IMRPhenomXAS, and 113 or 345 µs for IMRPhenomD, depending only on where the allocator put the arrays. `FrequencyGrid` now stores them in one allocation with staggered offsets, which keeps the timings stable.

Results

MODEL	RUST, PER CALL	RUST, GRID OR WORKSPACE	LAL	SPEED-UP, PER CALL	SPEED-UP, GRID
TaylorF2	232 µs	60 µs	237 µs	1.0×	3.9×
IMRPhenomD	188 µs	116 µs	392 µs	2.1×	3.4×
IMRPhenomXAS	171 µs	100 µs	354 µs	2.1×	3.6×
IMRPhenomXHM	1.38 ms	1.01 ms	2.30 ms	1.7×	2.3×
IMRPhenomXP, NNLO (102)	594 µs	177 µs	1.11 ms	1.9×	6.3×
IMRPhenomXP, MSA (223)	1.85 ms	1.01 ms	2.56 ms	1.4×	2.5×
IMRPhenomXP, SpinTaylor (320)	1.18 ms	804 µs	3.37 ms	2.9×	4.2×
IMRPhenomXPHM, NNLO (102)	3.37 ms	963 µs	7.54 ms	2.2×	7.8×
IMRPhenomXPHM, MSA (223)	8.19 ms	3.41 ms	14.56 ms	1.8×	4.3×
IMRPhenomXPHM, SpinTaylor (320)	4.65 ms	2.47 ms	12.66 ms	2.7×	5.1×
IMRPhenomXPNR	7.02 ms	4.01 ms	15.77 ms	2.2×	3.9×
NRSur7dq4 (time domain)	1.01 ms	1.00 ms	5.59 ms	5.5×	5.6×

Mean time per waveform over 300 random binaries evaluated in sequence, single thread: M log-uniform in [20, 150] $M_\odot$, q ∈ [1/6, 1], isotropic spins with $|\chi| \le 0.9$, isotropic inclination, distance 100–3000 Mpc, f_{ref} = 20 Hz. The frequency-domain models use 8,033 frequencies from 20 to 1024 Hz (Δf = 0.125 Hz), with LAL's multibanding off. NRSur7dq4 is in the time domain at 4096 Hz from the start of the surrogate; its "grid" column reuses one workspace. LAL is called through Python, which adds about 2 µs per call. Scripts: `report/serial_benchmark.py` and `examples/serial_benchmark.rs`.

What limits the gains

- **TaylorF2.** On the grid, each frequency costs one polynomial and the vectorised `sin_cos`, about 6 ns. Its per-call time depends noticeably on memory layout. It includes allocating the outputs, and it slows down when the allocator places them in a way that conflicts in the cache with the input. A call takes about 150 µs for a single repeated binary, but averaged more than that over these 300 draws, roughly level with LAL. The grid path writes into fixed buffers and avoids this.
- **IMRPhenomD, XAS and XHM.** Most of a 20–1024 Hz grid lies in the merger–ringdown region of a binary black hole, where each frequency still calls `atan` and `exp`. IMRPhenomXHM adds per-mode work. Its (3,2) ringdown is a sum of a spheroidal term and a rotated (2,2) term. The per-frequency path combines them with `cexp` and then splits the result back into amplitude and phase with `atan2` and `hypot`. The grid path converts the two terms separately, which removes those calls.
- **IMRPhenomXP and XPHM.** Their grid paths are 1.5×–3.5× faster than the per-call paths. They work in passes over the grid. The Euler angles are computed for each m′, the NNLO angles from the stored powers without transcendental functions, and the MSA angles in blocks of 32 frequencies. An MSA angle already computed for another m′ at a bit-identical argument is reused, which skips 29% of the evaluations for IMRPhenomXPHM. The ε phase is folded into each mode's phase before one vectorised conversion, and the twist-up keeps its sums in registers. What remains is the MSA angles, whose spin-cubic roots and corrections must use the C library to stay stable for nearly aligned spins; the SpinTaylor PN integration, about 0.5 ms; and the IMRPhenomXHM mode evaluations.
- **NRSur7dq4.** It has no frequency grid. Reusing a workspace removes the allocations. The workspace also keeps the last binary's inertial-frame modes. When only inclination, phase, distance or Δt change, the surrogate is skipped and a waveform costs 0.03–0.15 ms instead of about 1 ms.

Larger levers

Past these constant factors, the biggest gains for a sampler come from evaluating fewer frequencies:

- LAL's multibanding, not yet ported;
- relative binning, or heterodyned likelihoods, which need a few hundred frequencies instead of thousands.

Caching the waveform across changes of the extrinsic parameters, as NRSur7dq4 now does, would also help the frequency-domain precessing models. Finally, the per-frequency loops are independent, so batches of waveforms could be spread over threads.

8 Long, low-mass waveforms

Low-mass binaries stay in band for minutes, so their waveforms need fine frequency resolution and very large grids. The frequency-domain models (all except NRSur7dq4) were tested and timed on grids with Δf = 1/128 Hz, from 10 or 20 Hz up to 0.5–4 kHz. That is 259k–521k frequencies, for binaries of about 3–10 $M_\odot$.

Accuracy

Each reference stores LAL's output at about 1250 indices per case, including both ends of the band. Rust evaluates the full grid, because the SpinTaylor angles depend on the grid. The SpinTaylor angles of a ~900 s backward integration (2.3 million PN samples) are bit-identical to `SimIMRPhenomXPSpinTaylorAngles`.

MODEL	CASES	LARGEST GRID	PER-FREQUENCY PATH	GRID PATH
IMRPhenomD	4	521,729	6.8×10^{-11}	7.5×10^{-11}
IMRPhenomXAS	4	521,729	1.0×10^{-10}	1.4×10^{-10}
IMRPhenomXHM	4	521,729	1.8×10^{-16}	1.4×10^{-10}
IMRPhenomXP (102)	2	259,585	4.5×10^{-16}	4.2×10^{-11}
IMRPhenomXP (223)	2	259,585	5.0×10^{-16}	4.3×10^{-11}
IMRPhenomXP (320)	2	259,585	4.2×10^{-16}	4.3×10^{-11}
IMRPhenomXPHM (102)	2	259,585	5.1×10^{-16}	4.2×10^{-11}
IMRPhenomXPHM (223)	2	259,585	4.1×10^{-16}	4.3×10^{-11}
IMRPhenomXPHM (320)	2	259,585	4.2×10^{-16}	4.3×10^{-11}
TaylorF2	4	521,729	1.9×10^{-11}	4.5×10^{-11}

Largest relative error against LAL over all long cases (`tests/long_waveforms.rs`).

The inspiral phase reaches 1e4–4e4 rad at the lowest frequencies. A 50-digit evaluation of the IMRPhenomXAS inspiral phase shows that even LAL's own value is off by up to 9 ulp, with a bias of +0.8 ulp from `pow(Mf, 1.0/6)` using a rounded exponent. The fast per-frequency path is within 8 ulp of the exact value and the grid path within 11, so they differ from LAL by up to about 20 ulp. That explains the ~1e-10 errors above. The paths that repeat LAL's operation order (XHM and the precessing models) agree to ~1e-16. A compensated grid formulation brought the grid path to 7.5 ulp but barely changed the difference from LAL, so it was not adopted. The tests allow 64 ulp of the leading-order phase at f_{min}, three to five times the worst case seen.

Memory and speed

For a 3 $M_\odot$ binary, the SpinTaylor angles integrate the PN equations from about 10 Hz and produce 2.8 million samples. The port used to store all 14 variables per sample and copy them three more times, peaking at 850 MB of heap. It now keeps only the orbital frequency and $\hat{L}$ per sample, reserves its buffers to LAL's length estimate plus a small margin, and transforms the arrays in place, with bit-identical output. For IMRPhenomXPHM with SpinTaylor angles on 259k frequencies, the heap peak fell from 850 to 169 MB and a call from about 450 to 290 ms. On the largest grid, peak memory is 375 MB in Rust against 1.95 GB for LAL.

MODEL, 4 KHZ	LAL	RUST, PER CALL	RUST, GRID	LAL, DEFAULT MULTIBANDING	SPEED-UP OVER LAL
TaylorF2	16.79 ms	9.62 ms	3.57 ms	–	1.7×
IMRPhenomD	23.12 ms	10.75 ms	4.83 ms	–	2.2×
IMRPhenomXAS	27.00 ms	12.73 ms	6.51 ms	–	2.1×
IMRPhenomXHM	125.88 ms	76.25 ms	51.10 ms	49.14 ms	1.7×
IMRPhenomXP (102)	85.72 ms	44.70 ms	13.04 ms	–	1.9×
IMRPhenomXP (223)	199.28 ms	141.38 ms	77.48 ms	–	1.4×
IMRPhenomXP (320)	256.11 ms	103.10 ms	78.37 ms	–	2.5×
IMRPhenomXPHM (102)	474.48 ms	228.61 ms	75.70 ms	339.97 ms	2.1×
IMRPhenomXPHM (223)	1032.16 ms	608.13 ms	262.08 ms	352.99 ms	1.7×
IMRPhenomXPHM (320)	1373.78 ms	427.80 ms	297.54 ms	977.90 ms	3.2×

Best of 5 interleaved rounds, single thread, 521,729 frequencies (Δf = 1/128 Hz, 20 Hz–4 kHz). The aligned-spin models use a 1.6 + 1.4 $M_\odot$ binary; the precessing models use 1.8 + 1.3 $M_\odot$ with in-plane spins. The speed-up compares Rust per call with LAL. Data: `report/data/long_timings.json` (`report/long_benchmark.py`).

Rust is faster than LAL on every frequency sequence. With multibanding on, LAL beats the Rust per-call path for IMRPhenomXHM and for IMRPhenomXPHM with MSA angles; porting multibanding would change that.

9 New models

IMRPhenomXPHM needs IMRPhenomXAS for the (2,2) mode, IMRPhenomXHM for the other co-precessing modes, the SpinTaylorT4 equations with LAL's GSL integrator for the angles, and IMRPhenomXP's precession set-up. Each was ported and tested on its own before the next was built on it.

COMPONENT	FOLLOWS	PUBLIC API	NOTES
IMRPhenomXAS `imrphenomx/xas.rs` and shared modules	`IMRPhenomXASGenerateFD`, `IMRPhenomXSetWaveformVariables`, the phase and amplitude fits	`IMRPhenomXAS::new(M, q, χ1, χ2, D)`, `from_components`, `with_reference_frequency`, the `Waveform` trait	Default model versions. All coefficients are bit-exact. $\varphi_c = 2\varphi_{ref}$, as in IMRPhenomD.
IMRPhenomXHM `xhm.rs`, `hm_*.rs`	`IMRPhenomXHMGenerateFDOneMode`, the 122022 amplitude and 122019 phase releases	`IMRPhenomXHM::new`, `with_mode_array`, `polarizations(f, φ_ref, ι)`, `mode(l, m, f, φ_ref)`	Modes (2,±2), (2,±1), (3,±3), (3,±2) and (4,±4), including the (3,2) spheroidal–spherical mixing. No multibanding.
SpinTaylorT4 `spintaylor/`	`XLALSimInspiralSpinTaylorT4`, GSL's `rkf45` with standard control, and LAL's adaptive driver with Hermite resampling	`SpinTaylorT4::new`, `evolve_orbit(Δt, f_start, f_end)`	Bit-identical orbits, 3.3–4.5× faster than LAL.
IMRPhenomXP `xp.rs`, `precession.rs`, `spintaylor_angles.rs`, `msa.rs`, `wigner.rs`	`IMRPhenomXPGenerateFD`, `IMRPhenomXGetAndSetPrecessionVariables`, the NNLO, MSA and SpinTaylor angle routines	`IMRPhenomXP::new(&PrecessingBinary)`, `with_version`, `polarizations`, `polarizations_uniform`, `spin_taylor_angles`	PrecVersion 102, 223 and 320 (the crate default), with LAL's fallbacks.
IMRPhenomXHM release 122019 `hm_*.rs`	the original higher-mode amplitudes (`PhenomXHMReleaseVersion` 122019)	`WaveformOptions`, used by XPNR	Needed by IMRPhenomXPNR; compared with LAL separately.
IMRPhenomXPNR `xpnr.rs`, `pnr*.rs`	the `IMRPhenomXPNR` generator: XPHM with PNR angles, the tuned co-precessing model and the antisymmetric modes	`IMRPhenomXPNR::new`, the common interface	See section 12.
IMRPhenomXPHM `xphm.rs`	`XLALSimIMRPhenomXPHMFrequencySequence`, `IMRPhenomXPHMTwistUp`, `IMRPhenomX_Initialize_Euler_Angles`	`IMRPhenomXPHM::new`, `with_version`, `with_mode_array`, `polarizations`, `polarizations_uniform`	No multibanding. See section 10.

Precession-angle versions and fallbacks

`PrecessionVersion` selects LAL's `PhenomXPrecVersion`:

- `Nnlo` (102): next-to-next-to-leading-order PN angles.
- `Msa` (223): the multiple-scale-analysis angles of Chatziioannou et al. (2017).
- `SpinTaylor` (320): numerical angles from integrating the SpinTaylorT4 precession equations, continued analytically through merger and ringdown.

Like LALSimulation, the code changes method without telling the caller:

- 320 uses the NNLO angles when the in-plane spins are below 1e-7.
- 320 uses MSA when the SpinTaylor integration cannot be used. This happens when the backward integration is shorter than one coarse step, or when the PN solution starts above f_{min}. In the random draws it hits about half of the binaries above 150 $M_\odot$.
- MSA uses NNLO when the MSA system cannot be initialised.

One default differs. In lalsimulation 6.2.0, IMRPhenomXP and XPHM default to `PhenomXPrecVersion` 300, which resolves to the MSA angles (223). The crate defaults to 320, which is what was asked for. Pass `PrecessionVersion::Msa` to get LAL's default.

10 IMRPhenomXPHM with SpinTaylor angles

Usage:

```
use waveformrs::imrphenomx::{IMRPhenomXPHM, PrecessingBinary, PrecessionVersion};

let binary = PrecessingBinary {
    mass_1: 40.0, mass_2: 20.0,                // solar masses
    spin_1: [0.4, -0.3, 0.2], spin_2: [0.1, 0.5, -0.3],  // at f_ref, L along z
    luminosity_distance: 400.0,                // Mpc
    inclination: 0.4, phi_ref: 0.1, f_ref: 20.0,
};
let model = IMRPhenomXPHM::with_version(&binary, PrecessionVersion::SpinTaylor)?;
let (h_plus, h_cross) = model.polarizations(&frequencies)?;           // like ChooseFDWaveformSequence
let (h_plus, h_cross) = model.polarizations_uniform(0.25, 20.0, 1024.0)?; // like ChooseFDWaveform (no multibanding)
```

A call runs these steps in order, each a port of the LAL routine named:

1. **Waveform and precession set-up**
   The masses and spins are ordered as LAL orders them. The code then builds the aligned-spin waveform variables from the L-frame spin components and computes the precession quantities: χ_p, the J frame and the angle θ_{JN}, the polarisation rotation ζ, and spherical harmonics up to $\ell = 4$ (`IMRPhenomXGetAndSetPrecessionVariables`). XPHM uses $m'_{min} = 1$ and $m'_{max} = 4$, so the angles must cover $Mf' = 2Mf/m'$ down to $(f_{min} - 0.5\ \text{Hz})\cdot 2/4$.

2. **SpinTaylorT4 evolution**
   The orbit-averaged precession equations are integrated backwards and forwards from f_{ref} on a coarse grid, then on a fine grid at high frequency. The integrator is the GSL RKF45 adaptive driver as LAL uses it, with Hermite resampling to the output step (`IMRPhenomX_InspiralAngles_SpinTaylor`).

3. **α and cos β splines, remnant spin**
   L and the spins are transformed to the J frame. α (unwrapped) and cos β are splined against Mf with GSL-identical natural cubic splines. The precessing final spin is computed from the spins at the end of the integration (`PhenomXPFinalSpinMod` 4), which also updates f_{RING} and f_{DAMP} of the co-precessing modes.

4. **Merger–ringdown continuation**
   Analytic forms for α and β are matched to the splines at the end of the PN solution and continued through merger and ringdown (`alphaMRD_coeff`, `betaMRD_coeff`).

5. **γ from the minimal-rotation condition**
   $d\gamma/df = -\cos\beta\, d\alpha/df$ is integrated with Boole's rule on a uniform grid. Its spline is evaluated up to $\max(Mf_{max}, f_{RING} + 4 f_{DAMP})\cdot 2/m'_{min}$, and the values at the transition frequency are stored (`IMRPhenomX_Initialize_Euler_Angles`).

6. **Angles per mode**
   For each $m' \in \{1, 2, 3, 4\}$, α, cos β and γ are evaluated at $Mf' = 2Mf/m'$, falling back to the MRD forms beyond the splines. The $\varepsilon = -\gamma$ phase and the Wigner half-angle powers are computed once per m' and frequency.

7. **Co-precessing modes**
   The (2,2) mode comes from IMRPhenomXAS and the others from IMRPhenomXHM (`IMRPhenomXHMGenerateFDOneMode`), with the precession-modified final spin and reference phase. When $q = 1$ with equal aligned spins, the odd-m' modes are skipped, as in LAL.

8. **Twist-up and polarisation rotation**
   Each mode is rotated into the inertial frame with the Wigner d-matrices for $\ell = 2, 3, 4$ and summed into h_+ and $h_\times$ (`IMRPhenomXPHMTwistUp`). The result is then rotated by 2ζ.

Validation

The XPHM tests cover:

- the SpinTaylor, NNLO and MSA versions;
- uniform grids;
- restricted mode arrays;
- eight heavy binaries confirmed to make LAL fall back from SpinTaylor to MSA.

Every suite agrees with LAL to ≤ 7e-16 relative error. In the random draws all 200 binaries per version are compared directly; the results are in the accuracy table. The intermediate quantities (PN solution, splines, J frame, final spin, MRD coefficients, γ) were also compared bit for bit, using a small C harness linked against `liblalsimulation`'s internal functions during development.

11 NRSur7dq4

NRSur7dq4 (Varma et al. 2019, arXiv:1905.09300) is a time-domain surrogate for numerical-relativity simulations of generically precessing binary black holes. It is trained for $q \leq 4$ and $|\chi| \leq 0.8$, and can be used up to $q = 6$ and $|\chi| = 1$. It includes the modes $\ell \leq 4$ and covers about 4300 M before merger. The port reproduces `XLALSimInspiralPrecessingNRSurPolarizations` (as called by `SimInspiralChooseTDWaveform`),

`XLALSimInspiralPrecessingNRSurModes` (as called by `SimInspiralChooseTDModes`) and `XLALPrecessingNRSurDynamics` bit for bit on this platform, and runs 5.7× faster.

Data

- **The file LAL needs.** lalsimulation 6.2.0 reads `NRSur7dq4_v1.0.h5` (17.5 MB, Zenodo 10.5281/zenodo.14999310, md5 `ec4a0a9c6af39ad14dc91d42957144e9`).
- **Older copies.** Files named `NRSur7dq4.h5`, from lalsuite-extra or gwsurrogate, contain identical datasets. They lack the `CANONICAL_FILE_BASENAME` attribute that LALSuite ≥ 7.25 checks, so LAL rejects them; the Rust reader accepts both.
- **Where the file comes from.** The crate doesn't ship the file. It finds it through `NRSUR7DQ4_DATA` or `LAL_DATA_PATH`, and the tests skip when it is absent. Here it lives in the git-ignored `lal_data/` directory.

Reading HDF5 without a C library

The surrogate file uses only the oldest HDF5 features:

- a version-0 superblock and version-1 object headers;
- symbol-table groups (version-1 B-trees, symbol nodes and local heaps);
- 4776 contiguous, uncompressed `float64` and `int64` datasets.

`src/hdf5.rs` (912 lines) reads exactly that subset, and returns `Hdf5Error::Unsupported` for anything else. Every dataset matches h5py in shape, type and a 64-bit checksum. Loading the surrogate takes 7.88 ms; LAL's first call, which loads the data, takes about 80 ms. This avoids linking the C HDF5 library or adding a dependency, at the cost of not reading general HDF5 files.

The evaluation

1. **Fits**

   Every data piece is a polynomial in (log q, χ_{1x}, χ_{1y}, $\hat{x}$, χ_{2x}, χ_{2y}, χ_a). LAL rebuilds the effective spins, log q and a 22-entry power table for every fit. Here they are computed once per parameter point and shared by all fits at a node; factors of order zero are dropped, which is exact. The sums keep LAL's order of operations.

2. **Dynamics**

   The orbital phase, the quaternion of the coprecessing frame and both spins are integrated from the reference time over the surrogate's time nodes. As in LAL, the integration starts with RK4 using the half nodes and then switches to fourth-order Adams–Bashforth.

3. **Coorbital modes**

   Fits are evaluated at the empirical-interpolation nodes from the dynamics interpolated there, then multiplied by the basis matrix. The basis product uses OpenBLAS's fused `dgemv` ordering.

4. **Frames**

   The modes are rotated from the coorbital frame to the coprecessing frame by the orbital phase, then to the inertial frame by Wigner D-matrices built from the quaternions. Pairs of values are computed together in NEON registers.

5. **Output**

   The result is resampled onto the uniform output grid with GSL-identical cubic splines and summed into h_+ and $h_\times$ with ${}_{-2}Y_{\ell m}$. The epoch is rounded to nanoseconds, and labels are switched when $m_1 < m_2$, both as LAL does.

Matching LAL bit for bit took four things:

- the arm64 fused multiply-adds;
- the GSL spline and divided-difference routines, ported operation by operation;
- OpenBLAS's order of summation;
- Darwin's `__sincos_stret`, which clang substitutes where LAL calls `sin` and `cos` of the same angle. These sites were confirmed in the disassembly of `liblalsimulation`.

Accuracy and speed

The reference tests compare 19 polarization cases from `ChooseTDWaveform`, plus 6 cases where LAL raises an error. They cover:

- q from 1 to 6, with q = 7 under unlimited extrapolation;
- spins up to 0.99;
- $f_{ref} \neq f_{min}$, $f_{min} = 0$, and $m_1 < m_2$;
- restricted mode arrays.

They also compare all modes from `ChooseTDModes`, the dynamics, and 3612 spin-weighted harmonics. Every comparison is bit-identical, with matching epochs and lengths. So are all 200 random draws: $q \in [1, 6]$, $|\chi| \leq 0.99$, $M \in [20, 300]\ M_\odot$, sampling at 1–8 kHz.

M [$M_\odot$]	RATE [HZ]	F_{MIN} [HZ]	SAMPLES	LAL	RUST	RUST, WORKSPACE	RUST, EXTRINSIC CHANGE	LAL MODES	RUST MODES
60	4096	20	4,048	5.26 ms	1.01 ms	994 µs	60 µs	7.82 ms	1.42 ms
60	16384	20	16,191	5.92 ms	1.10 ms	1.08 ms	147 µs	10.95 ms	2.08 ms
20	4096	55	1,721	5.25 ms	994 µs	972 µs	43 µs	7.28 ms	1.31 ms
20	16384	55	6,882	5.40 ms	1.02 ms	1.01 ms	81 µs	9.25 ms	1.57 ms
100	4096	20	1,672	5.93 ms	1.00 ms	987 µs	41 µs	7.43 ms	1.28 ms
100	1024	20	418	5.88 ms	995 µs	978 µs	31 µs	6.99 ms	1.22 ms
200	2048	20	400	5.97 ms	1.00 ms	987 µs	30 µs	7.02 ms	1.21 ms
300	1024	20	202	6.02 ms	1.00 ms	988 µs	29 µs	6.90 ms	1.20 ms

Single thread, q = 2, generic spins. "Workspace" reuses buffers, alternating between two binaries so nothing is cached. "Extrinsic change" changes only the inclination, so the cached inertial modes are reused. Data: `report/data/nrsur_timings.json`, from `report/nrsur_benchmark.py`.

About half of the remaining time goes to the frame rotations. The dynamics fits, the coorbital fits with the basis product, and the trigonometric functions take most of the rest.

12 IMRPhenomXPNR

In lalsimulation 6.2.0, IMRPhenomXPNR is IMRPhenomXPHM with these settings forced on:

- the 2019 higher-mode release (122019, also ported);
- the PNR tuned Euler angles;
- the tuned co-precessing model;
- the antisymmetric (2,±2) contribution;
- `PhenomXPrecVersion` 330 and final-spin version 7.

The port (`src/imrphenomx/pnr*.rs`, `xpnr.rs`) follows LAL's many regime switches:

- PNR is off when the in-plane spins are below 1e-7;
- NNLO angles are used inside PNR below 1e-6;
- SpinTaylor falls back to MSA, and MSA to NNLO;
- outside the calibration region ($q \leq 8.5$, $\chi \leq 0.85$) the model blends into PN angles, reaching them at $q = 12$ and $\chi = 1.2$;
- the co-precessing tuning tapers off for $10 < q \leq 20$.

LAL's PNR angles are among those affected by the OpenMP race (section 16), so every reference was generated single-threaded.

The reference suites contain 48 cases covering every switch above, uniform grids, restricted mode arrays and $m_1 < m_2$. They also include the J-frame modes of `SimInspiralChooseFDModes`, which supports XPNR. All agree with LAL to ≤1.3e-15 relative. So do all 200 random draws in the accuracy table (section 5). It takes 3.86 ms against 9.41 ms for LAL on the benchmark binary, and 7.35 ms for LAL with multibanding on a uniform grid.

The parity fix of lalsuite master

Merge request !2593 (“Fixing inconsistency in antisymmetric phase for XO4a/XPNR that affects parity symmetry”) was merged into lalsuite master on 22 September 2026, after lalsimulation 6.2.1. Neither 6.2.0, which this crate reproduces, nor 6.2.1 has it. The fix changes the phase of the antisymmetric (2,2) mode in two places:

- Below the transition frequency Mf_T, the phase is $\phi_{22}/2 + \alpha + A_0 Mf + \phi_{A0}$. In 6.2.0, ϕ_{A0} is `alpha_offset` $= \alpha(f_{ref}) - \alpha_0$, and $\alpha_0 = \pi - \kappa$ is the azimuth of the line of sight about J. The fix uses $\phi_{A0} = -\alpha(f_{ref}) - \pi/2$, with α at f_{ref} evaluated in the same regime as at Mf_T (calibration region, transition or PN).
- The polarization angle ζ added to the whole phase becomes the azimuth `phiJ_Sf` of J in the L frame.

Both 6.2.0 offsets depend on where the observer is, and neither changes sign when the binary is reflected in its orbital plane. Under that reflection, the in-plane spins reverse and the line of sight goes from inclination ι to $\pi - \iota$. Physically the waveform should then become $(h_+, -h_\times)$. In 6.2.0 it doesn't, by 2e-3 to 1e-1 of the peak amplitude, because the antisymmetric mode keeps its sign. The azimuth of J shifts by π under the reflection, which flips the sign of the antisymmetric mode as required. The patch also builds the rotation of the evolved spins into the L frame from L's components rather than from `atan2` and `acos`. That is the same rotation mathematically, but it commutes exactly with the reflection in floating point. It moves the ill-conditioned (3,2) mode by up to ~1e-7.

The crate applies the fix by default. `IMRPhenomXPNR::with_parity_fix(false)` or `WaveformOptions::pnr_parity_fix = false` reproduce lalsimulation 6.2.0 and 6.2.1, and so does `WaveformOptions::lalsimulation_defaults()`. The 6.2.0 reference suites above are run that way. The fixed model is validated three ways:

- **Against a patched LAL.** `scripts/build_patched_lalsimulation.sh` builds lalsimulation 6.2.0 with the patch and the flags of the conda-forge build. Built with conda-forge's clang 19 and without the patch, it reproduces the conda-forge library bit for bit. The four reference suites from the patched build (`tests/data/imrphenomxpnr_parityfix*.json`) agree with the crate to $\lesssim$1.2e-15. So do 200 random draws: largest relative error 3.2×10^{-15}, largest mismatch 2.2×10^{-16}. The JSON files record the build, "UNCLEAN: lalsimulation-v6.2.0 + lalsuite!2593 (git diff 64d2004def^1 64d2004def -- lalsimulation/lib)".
- **Against LAL's own unit test.** `test_phenomX.py` on lalsuite master stores amplitude and phase statistics of the fixed model, with tolerances of 1e-3 and 3e-3. The crate's values are within 3.1e-4 (main modes) and 4.6e-4 ((3,2) mode) of them. The difference has two sources. The test leaves LAL's multibanding on, which moves its statistics by up to 2.4e-4 and 4.6e-4. Also, one h_+ phase statistic depends on a small last sample that sits at the $\pm\pi$ branch of `np.unwrap`. With multibanding off, the patched LAL and the crate agree to ~1e-15 (`lal_unit_test_of_lalsuite_master` in `tests/xpnr.rs`).
- **The mirror symmetry itself.** The test covers seven binaries spanning the calibration, transition, PN and MSA-fallback regions, plus a uniform grid. The fixed model satisfies the mirror relation to 2e-15–6e-13; without the fix it is violated by 2e-3–1e-1. LAL behaves the same way: violations at that level in 6.2.0, ~1e-14 with the patch.

Nothing else in lalsuite master changes XPNR's default output. The other changes since 6.2.0 are testing-GR terms that are zero by default, licence headers, conversions of polar spins, and small changes to tolerances and the integrator in the eccentric and SpinTaylor code. None of these alter XPNR's output: a build of upstream master (6d2217a745, with master's lal) is bit-identical to 6.2.0 plus the patch on all four suites and on the unit-test statistics.

Grid path

`polarizations_on_grid` keeps the set-up of the sequence path. It then evaluates each co-precessing mode in passes over the grid:

1. the PNR angles, which depend on the mode through the frequency map;
2. the twisting-up factors;
3. the mode itself, from the stored powers of the grid;
4. the antisymmetric part.

Amplitude and phase are converted in one vectorised pass. Over 300 random binaries on the 8033-point grid, it takes 4.01 ms per waveform against 7.02 ms for the sequence path (1.8×) and 15.77 ms for LAL. It agrees with the sequence path to ~1e-13, with and without the fix. Most of the remaining time goes to the SpinTaylor integration and the PNR angle interpolants (~0.9 ms), and to LAL's O(N·80) moving average in the antisymmetric amplitude (~0.3 ms). The moving average keeps LAL's operations but skips windows whose inputs are bit-identical to the previous window's. Above the ringdown frequency, that is most of them. Both paths share this, so their output is unchanged.

13 Using the crate

One interface for every model

All models share one set of parameters, `Binary`, and two traits, in the conventions of `SimInspiralChooseFDWaveform`:

- `Binary` holds masses in $M_\odot$, dimensionless spin vectors at f_{ref}, distance in Mpc, inclination, orbital ϕ_{ref} and f_{ref}. `PrecessingBinary` is now an alias for it.
- `FrequencyDomainModel` is implemented by TaylorF2, IMRPhenomD, XAS, XHM, XP, XPHM and XPNR.
- `TimeDomainModel` is implemented by NRSur7dq4.
- `choose_fd_model`, `choose_td_model` and `choose_fd_modes` dispatch by an `Approximant` enum that uses LAL's names. A `WaveformOptions` struct carries the LALDict-style settings.

```
use waveformrs::{choose_fd_model, Approximant, Binary, FrequencyDomainModel, WaveformOptions};

let options = WaveformOptions::default()             // or ::lalsimulation_defaults()
    .with_precession_version(PrecessionVersion::Msa)
    .with_version_fallback(false);                   // error instead of switching versions
let model = choose_fd_model(Approximant::IMRPhenomXPHM, &binary, &options)?;
let (hp, hc) = model.polarizations(&frequencies)?;  // = SimInspiralChooseFDWaveformSequence
model.polarizations_on_grid(&grid, &mut hp, &mut hc)?;
let modes = model.modes(&frequencies)?;             // h_lm without the spherical harmonics
```

- **Reference phase.** `phi_c` in the `Waveform` trait now means LAL's orbital reference phase for every model. IMRPhenomD and IMRPhenomXAS used to expect twice that. Their output is bit-identical to the old calls with $2\phi_{\rm ref}$. This is the one breaking change, recorded in `CHANGELOG.md`.
- **Checks.** `from_binary` applies LAL's checks: aligned-spin models reject in-plane spins, `f_ref = 0` means LAL's default, and the bodies are ordered heavier first.
- **Precession default.** The crate keeps SpinTaylor (320) as its default precession version; `WaveformOptions::lalsimulation_defaults()` selects LAL's MSA (223).

Modes without the spherical harmonics

Among the ported models, `SimInspiralChooseFDModes` in 6.2.0 supports IMRPhenomXHM, IMRPhenomXPHM and IMRPhenomXPNR. For these, `modes()` reproduces it to ≤4e-16:

- a two-sided uniform grid;
- for XPHM and XPNR, J-frame modes for all ℓ, m, with each mode set up as LAL sets it up.

Summing XPHM's SpinTaylor modes reproduces its polarizations only to ~4e-6. That difference is LAL's own: `ChooseFDModes` gives each J-frame mode its own range of angle integration. For TaylorF2, D, XAS and XP, which LAL doesn't support, the (2,±2) mode (or XP's J-frame (2,m) modes) is returned in a form consistent with the polarizations.

Switching off automatic version changes

By default IMRPhenomXP, XPHM and XPNR change their precession version as LAL does:

- to NNLO when the in-plane spins are tiny;
- to MSA when the SpinTaylor angles can't be computed;
- to NNLO when the MSA system fails.

`with_version_fallback(false)` (or the option) returns `PrecessionVersionUnavailable` instead. Tests cover each fallback both ways.

Batched evaluation

`Batch::new(threads)` evaluates many binaries on one grid, or NRSur7dq4 in the time domain, writing into rows of caller-owned buffers. It uses standard-library scoped threads, with one set of buffers or workspace per thread. The results don't depend on the thread count, which the tests check bit for bit.

MODEL	1 THREAD	2 THREADS	4 THREADS	8 THREADS	24 THREADS
IMRPhenomXAS	6,729	15,517 (2.3×)	30,234 (4.5×)	60,049 (8.9×)	125,126 (18.6×)
IMRPhenomXHM	1,099	2,106 (1.9×)	4,071 (3.7×)	7,800 (7.1×)	17,853 (16.2×)
IMRPhenomXP (SpinTaylor)	1,278	2,410 (1.9×)	4,604 (3.6×)	8,694 (6.8×)	19,340 (15.1×)
IMRPhenomXPHM (SpinTaylor)	435	821 (1.9×)	1,536 (3.5×)	2,918 (6.7×)	6,666 (15.3×)
IMRPhenomXPHM (MSA)	306	582 (1.9×)	1,097 (3.6×)	2,028 (6.6×)	4,564 (14.9×)
IMRPhenomXPNR	283	538 (1.9×)	955 (3.4×)	1,706 (6.0×)	4,021 (14.2×)
NRSur7dq4 (time domain)	1,007	1,928 (1.9×)	3,583 (3.6×)	6,644 (6.6×)	14,738 (14.6×)

Waveforms per second on 8,033 frequencies (20–1024 Hz), with the speed-up over one thread in brackets. Apple M2 Ultra (16 performance and 8 efficiency cores), which is why scaling flattens above 16 threads. `examples/batch_benchmark.rs`.

Between the time and frequency domains

`conversions::simulate_td` and `simulate_fd` reproduce LAL's `SimInspiralTD` and `SimInspiralFD` for every approximant. They include:

- the duration and frequency bounds;
- the Planck and Tukey tapers;
- LAL's Butterworth high-pass filter (bit-exact for identical input);
- the Stage 1/2 conditioning;
- the epochs, to the nanosecond.

The FFTs come from `rustfft`, behind the default `fft` feature. Epochs, lengths and sampling intervals match LAL exactly. The values match to:

- ~4e-16 for NRSur7dq4 into the frequency domain;
- ≤4e-13 for the frequency-domain re-conditioning of FD models;
- ~2e-12 for the FD models into the time domain;
- 3.6e-11 for a 68 s TaylorF2 neutron-star binary, a limit set by the model's own phase rounding.

The time-domain floor comes from LAL's high-pass filter. Its IIR sections have poles near z = 1 and amplify last-bit differences of their input by ~1e4, so differences between `rustfft` and FFTW can't be removed.

LAL behaviour reproduced here:

- For IMRPhenomD, XAS and XHM, `SimInspiralTD` conditions twice: once in `TDFromFD`, then again in the time domain.
- `ChooseFDWaveform` rejects NRSur7dq4; only `SimInspiralFD` converts it.

CONVERSION	MODEL	RUST	LAL	SPEED-UP
SimInspiralTD	IMRPhenomD 36+29 Msun	884 µs	2.34 ms	2.7×
SimInspiralTD	IMRPhenomXAS 36+29 Msun	932 µs	2.38 ms	2.6×
SimInspiralTD	IMRPhenomXHM 36+29 Msun	1.49 ms	3.33 ms	2.2×
SimInspiralTD	IMRPhenomXP 36+29 Msun	1.58 ms	4.74 ms	3.0×
SimInspiralTD	IMRPhenomXPHM 36+29 Msun	4.17 ms	11.52 ms	2.8×
SimInspiralTD	IMRPhenomD 10+5 Msun, 10 Hz	32.29 ms	90.56 ms	2.8×
SimInspiralTD	TaylorF2 1.4+1.3 Msun, 30 Hz	23.49 ms	58.72 ms	2.5×
SimInspiralFD	NRSur7dq4 40+20 Msun, df = 0, 2048 Hz	1.04 ms	5.81 ms	5.6×
SimInspiralFD	NRSur7dq4 40+20 Msun, df = 1/8, 2048 Hz	1.24 ms	6.27 ms	5.1×
forward real FFT, N = 2^16	plan + FFT	658 µs	496 µs	0.8×
reverse real FFT, N = 2^16	plan + FFT	650 µs	782 µs	1.2×
forward real FFT, N = 2^16	cached plan	197 µs	496 µs	2.5×
forward real FFT, N = 2^20	plan + FFT	13.52 ms	11.75 ms	0.9×
reverse real FFT, N = 2^20	plan + FFT	13.34 ms	17.34 ms	1.3×
forward real FFT, N = 2^20	cached plan	4.13 ms	11.75 ms	2.8×

Single thread. `examples/conversions.rs` and `scripts/time_conversions_lal.py`; data in `report/data/conversion_timings.json`.

14 Python bindings

The Python package `waveformrs`, in `python/`, exposes the common interface to Python with numpy arrays in and out. It is built with PyO3 and maturin as a separate crate that depends on the main one. One abi3 wheel serves CPython 3.9 and later, with numpy 1.21 or later.

```
import numpy as np
import waveformrs as wf

frequencies = np.arange(20.0, 1024.0, 0.125)
hp, hc = wf.polarizations("IMRPhenomXPHM", frequencies, mass_1=36.0, mass_2=29.0,
                          spin_1=(0.4, 0.2, 0.3), f_ref=20.0, precession_version=223)

grid = wf.FrequencyGrid.uniform(20.0, 1024.0, 0.125)    # powers precomputed once
parameters = dict(mass_1=m1, mass_2=m2, spin_1=s1, spin_2=s2)   # arrays, one row per binary
result = wf.batch_polarizations("IMRPhenomXPNR", grid, parameters, threads=8)
hplus, hcross = result                                  # (n_binaries, n_frequencies)
result.ok, result.errors                                # per-binary failures

h = wf.td_polarizations("NRSur7dq4", 1 / 4096, 20.0, mass_1=40.0, mass_2=20.0)
```

- **Parameters.** Binaries are given as keyword arguments with the crate's names and LAL's units: `mass_1`, `mass_2`, `spin_1`, `spin_2`, `luminosity_distance` (Mpc), `inclination`, `phi_ref`, `f_ref`. Options are given in the same keywords or as a `WaveformOptions` object: `precession_version`, `mode_array`, `version_fallback`, `pnr_parity_fix`. Approximants use LAL's names.
- **Frequency domain.** `polarizations` corresponds to `SimInspiralChooseFDWaveformSequence`. `FrequencyGrid` and `polarizations_on_grid` give the grid paths. `modes` and `choose_fd_modes` give the modes in LAL's two-sided layout.
- **Time domain.** `td_polarizations` and `td_modes` cover NRSur7dq4, with one workspace per thread, and the frequency-domain models through `SimInspiralTD`. `simulate_td` and `simulate_fd` are the conversions. Results carry their epoch and sampling.
- **Batches.** `batch_polarizations` and `batch_td_polarizations` take a dict of arrays, a structured array or an (n, 12) table. They run on `threads` threads with the GIL released and write straight into numpy-owned arrays. A binary that fails, even by a panic, is reported in `ok` and `errors` without stopping the batch, unless `strict=True`. `exact=True` uses the sequence path, so each row is bit-identical to `polarizations`.
- **Errors.** The crate's `Error` becomes a hierarchy under `WaveformError`. Its classes also derive from `ValueError`, `RuntimeError` or `OSError`, so generic handlers still work. The package has docstrings and a `.pyi` stub.

The 86 pytest tests check several things. TaylorF2, IMRPhenomD, XAS, XHM, XP and XPHM (each precession version) and XPNR (without the parity fix) agree with lalsimulation's Python API at the crate's tolerances; NRSur7dq4 agrees bit for bit. Batches are bit-identical for every thread count and equal to single calls. Each error class is raised where it should be. Python threads keep running while a call is in progress. The XPNR parity fix restores the mirror symmetry. The tests found a crate bug: IMRPhenomXHM panicked, rather than returning an error, when `f_ref = 0` and the first frequency was negative. It is fixed, and a crate test now checks that no model panics on frequencies LAL would refuse.

MODEL	WAVEFORMRS	LALSIMULATION
TaylorF2	3 µs	3 µs
IMRPhenomD	2 µs	45 µs
IMRPhenomXAS	3 µs	6 µs

Cost of one Python call on 4 frequencies, where the call, argument conversion and model set-up dominate.

MODEL	POLARIZATIONS	ON_GRID	LALSIMULATION	SPEED-UP
TaylorF2	164 µs	53 µs	333 µs	6.3×
IMRPhenomD	204 µs	117 µs	494 µs	4.2×
IMRPhenomXAS	266 µs	142 µs	465 µs	3.3×
IMRPhenomXHM	1.44 ms	1.04 ms	2.53 ms	2.4×
IMRPhenomXP (320)	1.12 ms	720 µs	3.09 ms	4.3×
IMRPhenomXPHM (102)	3.66 ms	1.22 ms	7.67 ms	6.3×
IMRPhenomXPHM (223)	9.06 ms	3.88 ms	15.50 ms	4.0×
IMRPhenomXPHM (320)	4.76 ms	2.45 ms	10.88 ms	4.4×
IMRPhenomXPNR	5.97 ms	2.78 ms	12.35 ms	4.4×
NRSur7dq4 (time domain)	997 µs	–	5.72 ms	5.7×

One waveform per call from Python, on 8033 frequencies (20–1024 Hz, Δf = 1/8 Hz) and one thread. NRSur7dq4 is time domain at 4096 Hz from 20 Hz. LAL uses `SimInspiralChooseFDWaveformSequence` with multibanding off, converting its arguments as a Python user would. The speed-up uses the faster waveformrs path. arm64 Darwin 25.6.0, 24 CPUs; medians of 7 repeats; `python/benchmarks/benchmark.py`.

MODEL	LAL LOOP	1 THREAD	4 THREADS	8 THREADS	ALL THREADS
IMRPhenomXAS	2,084	6,585	18,284	25,944	35,096
IMRPhenomXHM	391	931	3,261	6,211	11,213
IMRPhenomXPHM (223)	65	258	949	1,863	3,765
IMRPhenomXPHM (320)	93	402	1,457	2,727	5,104
IMRPhenomXPNR	77	331	1,171	2,258	4,302
NRSur7dq4 (time domain)	171	1,004	3,648	7,204	13,064

Waveforms per second for batches of 64 binaries from Python, against a Python loop over lalsimulation (one thread).

Packaging. `maturin develop --release` in `python/` installs the package into the active environment, and `maturin build --release` builds a wheel of ~1.2 MB. The wheel was also tested with the system Python 3.9 and numpy 1.26 and 2.0. The source distribution bundles the crate as cargo packages it, which now leaves out the reference data, report and scripts (0.5 MB instead of 14.6 MB). Nothing is on PyPI, and the bindings have only run on macOS arm64.

15 Other platforms

`docker/` builds Linux containers for arm64 (native) and amd64 (emulated with Rosetta). Each has conda-forge's lalsimulation 6.2.0 and a Rust toolchain; `docs/linux.md` explains how to run them. The test suite passes on all three platforms, against both the committed macOS references and each container's own LAL.

	MACOS ARM64	LINUX ARM64	LINUX AMD64 (EMULATED)
Tests against the committed references	`strict`	`pass`	`pass`
Tests against that platform's LAL	–	`pass`	`pass`
Without the `fft` feature	`pass`	`pass`	`pass`
Multiply-add mirroring	clang's contraction	clang's subset of GCC's	none (baseline x86-64)

What differs on Linux

- **Cube roots.** On Linux, Rust's `f64::cbrt` is not glibc's. Rust's compiler-builtins links a correctly rounded `cbrt` ahead of libm, and it differed from the one LAL calls on 57% of arguments. This was the largest source of Linux differences. `src/cmath.rs` ports glibc's algorithm, which is tested bit for bit against glibc on two million arguments and is also 10–20% faster.
- **Fused multiply-adds.**
  - On aarch64, GCC's `-ffp-contract=fast` fuses a superset of what clang fuses. Mirroring clang's subset still brings Linux arm64 closer to its LAL (median IMRPhenomXHM error 1e-8 against 5e-7 without fusing).
  - The linux-64 builds target `nocona` and contain no FMA instructions, so nothing is fused on x86-64.
  - The vectorised maths no longer falls back to a software `fma` on x86-64.
  - A `--cfg waveformrs_fma` flag overrides the choice.
- **Test tolerances.** The strict tolerances apply only on macOS arm64. Elsewhere only the suites that amplify last-bit differences are loosened, each to about ten times the observed difference: the XHM (3,2) mode, MSA, SpinTaylor, the conversions and the long waveforms. `WAVEFORMRS_REFERENCE_DATA` points the tests at references generated locally.

LAL itself isn't portable at that level. Comparing its own outputs between builds:

REFERENCE SUITE	LAL LINUX ARM64 VS MACOS	LAL LINUX AMD64 VS MACOS
`choose_fd_modes`	7.0×10^{-4}	1.2×10^{-3}
`imrphenomd`	1.1×10^{-12}	8.9×10^{-13}
`imrphenomxas`	2.3×10^{-12}	3.0×10^{-12}
`imrphenomxhm`	3.5×10^{-6}	4.0×10^{-5}
`imrphenomxhm_modes`	1.1×10^{-3}	2.2×10^{-3}
`imrphenomxp_msa`	3.6×10^{-9}	3.2×10^{-9}
`imrphenomxp_nnlo`	3.1×10^{-13}	3.1×10^{-13}
`imrphenomxp_spintaylor`	7.9×10^{-11}	6.2×10^{-12}
`imrphenomxp_spintaylor_angles`	9.0×10^{-11}	1.8×10^{-11}
`imrphenomxp_spintaylor_fallback`	1.4×10^{-13}	4.3×10^{-13}
`imrphenomxp_uniform`	7.1×10^{-11}	5.3×10^{-12}
`imrphenomxphm_modes`	1.3×10^{-4}	1.2×10^{-4}
`imrphenomxphm_msa`	3.0×10^{-6}	3.5×10^{-5}
`imrphenomxphm_nnlo`	8.0×10^{-6}	1.9×10^{-5}
`imrphenomxphm_spintaylor`	2.2×10^{-5}	9.0×10^{-6}
`imrphenomxphm_spintaylor_fallback`	8.7×10^{-6}	1.6×10^{-5}
`imrphenomxphm_uniform`	1.9×10^{-5}	1.4×10^{-5}
`nrsur7dq4`	3.1×10^{-14}	7.0×10^{-13}
`nrsur7dq4_dynamics`	1.1×10^{-12}	4.8×10^{-13}
`nrsur7dq4_modes`	4.1×10^{-14}	8.2×10^{-13}
`spin_weighted_harmonics`	0	0
`spintaylor`	7.5×10^{-10}	1.3×10^{-9}
`taylorf2`	6.8×10^{-12}	6.8×10^{-12}

Largest relative difference between LAL's own outputs on the committed reference cases. Data: `report/data/linux_reference_comparison.json`.

MODEL	LINUX ARM64: MEDIAN / MAX REL. ERROR	LINUX AMD64: MEDIAN / MAX REL. ERROR
IMRPhenomD	6.4×10^{-13} / 4.0×10^{-11}	6.1×10^{-13} / 5.7×10^{-11}
IMRPhenomXAS	1.0×10^{-12} / 6.2×10^{-11}	7.0×10^{-13} / 1.0×10^{-10}
IMRPhenomXHM	9.7×10^{-9} / 1.6×10^{-5}	0 / 3.8×10^{-7}
IMRPhenomXP	1.1×10^{-12} / 9.3×10^{-7}	3.7×10^{-16} / 2.1×10^{-15}
IMRPhenomXPHM	3.7×10^{-7} / 1.3×10^{-4}	3.8×10^{-16} / 4.8×10^{-7}
IMRPhenomXPHM_MSA	3.9×10^{-8} / 2.0×10^{-5}	3.9×10^{-16} / 5.5×10^{-7}
IMRPhenomXPHM_NNLO	1.4×10^{-8} / 8.8×10^{-6}	3.8×10^{-16} / 3.4×10^{-7}
IMRPhenomXP_MSA	6.8×10^{-14} / 9.4×10^{-9}	3.7×10^{-16} / 1.8×10^{-15}
IMRPhenomXP_NNLO	3.5×10^{-14} / 2.9×10^{-12}	3.6×10^{-16} / 1.6×10^{-15}
NRSur7dq4	1.3×10^{-14} / 1.5×10^{-13}	5.5×10^{-16} / 1.0×10^{-15}
TaylorF2	7.8×10^{-13} / 4.9×10^{-11}	9.8×10^{-13} / 6.2×10^{-11}

waveformrs against the same container's lalsimulation, random draws as in section 4. Data: `report/data/linux_validation_{arm64,amd64}.json`.

MODEL	RUST	LAL	SPEED-UP	RUST GRID PATH
TaylorF2	164 µs	215 µs	1.3×	42 µs
IMRPhenomD	177 µs	295 µs	1.7×	119 µs
IMRPhenomXAS	177 µs	197 µs	1.1×	118 µs
IMRPhenomXHM	1.16 ms	1.71 ms	1.5×	1.20 ms
IMRPhenomXP (102)	404 µs	588 µs	1.5×	318 µs
IMRPhenomXP (223)	1.23 ms	1.54 ms	1.3×	1.40 ms
IMRPhenomXP (320)	864 µs	2.59 ms	3.0×	950 µs
IMRPhenomXPHM (102)	2.03 ms	4.03 ms	2.0×	1.32 ms
IMRPhenomXPHM (223)	5.27 ms	8.80 ms	1.7×	4.41 ms
IMRPhenomXPHM (320)	3.52 ms	7.59 ms	2.2×	2.80 ms
NRSur7dq4	1.10 ms	5.59 ms	5.1×	–

Linux arm64 container on the same machine, single thread. The aligned-spin models run 1.3–1.7× slower per call than on macOS because glibc's `sincos`, `cbrt` and `atan` are slower than Apple's. Emulated amd64 timings aren't meaningful and are not shown. Data: `report/data/linux_timings_arm64.json`.

Not covered: real x86-64 hardware, and builds with AVX enabled, which give wrong results under Rosetta even with unchanged arithmetic.

16 Issues found in LALSimulation

OpenMP data race in IMRPhenomXP (MSA and PNR angles)

`IMRPhenomXPGenerateFD` evaluates the waveform in a `#pragma omp parallel for` loop (`LALSimIMRPhenomX.c:2281` on master). `IMRPhenomX_Return_phi_zeta_costhetaL_MSA` writes the per-frequency spin roots and spin magnitude into the shared `pPrec` struct (`LALSimIMRPhenomX_precession.c:2262–2273`) and then reads them back; the PNR angles are passed through `pPrec` in the same way. With more than one thread, some frequencies are computed with another frequency's values. Results:

- **Default IMRPhenomXP (300, which resolves to MSA):** differences of 4–8% of the peak amplitude between runs.
- **320 falling back to MSA:** 14–21%.
- **PNR angles:** errors of order one.
- **NNLO (102), SpinTaylor (320) and IMRPhenomXPHM:** unaffected.

Single-threaded output is correct and reproducible. The bug is still present on lalsuite master (`ba91eb79ca`). The full report, with a reproducer and the lines to change, is `report/lalsimulation-openmp-race.md` (with a PDF copy), and the reproducer script is `scripts/reproduce_lal_openmp_race.py`.

Smaller findings

- **Error state left set after a successful fallback.** When the SpinTaylor integration fails through `XLAL_ERROR`, the C drivers fall back to MSA and return a valid waveform, but `xlalErrno` stays set. The Python bindings then raise "Internal function call failed" for a call that succeeded.
- **Hidden global initialisation.** `XLALSimIMRPhenomXPSpinTaylorAngles` reads the global `powers_of_lalpi`, which only the waveform drivers initialise. Called first in a fresh process, its 3PN orbital angular momentum misses the π^2 term, and the angles are off by ~1e-3.
- **Ill-conditioned (3,2) set-up.** The IMRPhenomXHM (3,2) intermediate phase is fixed by a finite-difference second derivative with step 1e-7. Moving the inputs by 1e-15 changes LAL's own (3,2) mode by up to 3e-3 and the polarizations by ~6e-5, a mismatch of ~2e-9.
- **A biased power.** LAL computes $Mf^{1/6}$ as `pow(Mf, 1.0/6)`, whose rounded exponent biases the IMRPhenomXAS inspiral phase by +0.8 ulp on average. On long inspirals LAL's phase is up to 9 ulp from the exact value.
- **Modes and polarizations disagree.** For IMRPhenomXPHM with SpinTaylor angles, the modes of `SimInspiralChooseFDModes` sum to its polarizations only to ~4e-6, because each J-frame mode gets its own range of angle integration.
- **NRSur7dq4 in the frequency domain.** `SimInspiralChooseFDWaveform` rejects NRSur7dq4; only `SimInspiralFD` converts it.
- **A sensitive filter.** LAL's high-pass filter amplifies last-bit input differences by ~1e4, which limits how reproducible its time-domain conditioning is.
- **Results depend on the platform.** LAL's arithmetic depends on how the compiler contracts multiply-adds, so SpinTaylor-based results differ between arm64 and x86-64 builds at about the 1e-7 level. This was later measured in Linux containers (section 15). LAL's own Linux and macOS builds differ by up to ~1e-3 in the IMRPhenomXHM (3,2) mode, and by ~1e-10 in the SpinTaylor angles.

17 API review: done and still open

The first review proposed nine changes. Five are now done:

- **One meaning for `phi_c`:** LAL's orbital reference phase throughout. This is the one breaking change; see `CHANGELOG.md`.
- **One polarization interface:** `FrequencyDomainModel` / `TimeDomainModel` with `choose_fd_model`, and a single `Error` type.
- **Caller-owned buffers:** `polarizations_into`, the grid paths and NRSur7dq4's workspace.
- **Optional parallelism:** `Batch`, without a new dependency.
- **The precession default:** documented, with `WaveformOptions::lalsimulation_defaults()`.

Still open:

1. **Private fields and consistent units.** `total_mass`, `luminosity_distance` (stored in metres, passed in Mpc) and `phasing` are still public and mutable on TaylorF2 and IMRPhenomD. This prevents caching anything that depends on them.
2. **Constructors that panic.** `IMRPhenomXAS::new` and `IMRPhenomXHM::new` still panic on invalid input. `from_binary` and `from_components` return errors, and samplers should use those.
3. **Method name clashes.** Some models have their own methods with the trait's names but different arguments, such as `IMRPhenomXHM::polarizations(f, φ, ι)`. The model's method takes precedence, so the trait methods have to be called through generics or `dyn`. Renaming the old methods in a future breaking release would remove the surprise.
4. **Set-up that depends on the grid.** As in LAL, the SpinTaylor angles depend on the grid through f_{min}, and $f_{ref} = 0$ means "the first frequency". Taking f_{min} explicitly would let the integration be reused across grids, as heterodyned and relative-binning likelihoods need.
5. **Multibanding.** LAL's default multibanding makes IMRPhenomXHM, XPHM and XPNR 1.1–1.7× faster on uniform grids. It is the largest remaining speed-up for full-band analyses, and the only case where LAL is faster than the Rust per-call path.

18 Limitations and future work

- **Multibanding is not implemented.** Rust reproduces LALSimulation with `PhenomXHMThresholdMband = PhenomXPHMThresholdMband = 0`. Comparisons with LAL's defaults on uniform grids will differ at the level of the multibanding tolerance.
- **Only the default model versions are ported.** Other `PhenomXPrecVersion` values (101, 103, 104, 220–222, 224) and the NRTidal variants are not ported. The PNR angles and antisymmetric modes exist only as part of IMRPhenomXPNR, not as options for IMRPhenomXP or XPHM. For the MSA family, 222 differs from 223 only by raising an error instead of falling back to NNLO; 220, 221 and 224 use different expressions.
- **Platforms.** The tests pass on macOS arm64 and in Linux arm64 and amd64 containers (section 15). Bit-for-bit agreement holds only against the macOS clang builds of LAL. Matching GCC's broader contraction of multiply-adds is not attempted. Nothing has run on real x86-64 hardware or with AVX enabled.
- **Only one LAL release was tested.** All reference data comes from lalsimulation 6.2.0. The exception is IMRPhenomXPNR with the parity fix, whose references come from 6.2.0 with the patch of lalsuite !2593, checked against a build of lalsuite master. When a lalsimulation release includes the fix, its references should be regenerated from that release, and the fix-off option kept only for older releases. Future LAL changes would show up as test failures, which is intended.
- **Grid paths are not bit-exact.** The `*_on_grid` methods agree with LALSimulation and with the per-frequency paths to about 1e-13, not bit for bit, because products of stored powers are rounded differently from direct powers. Use the per-frequency methods where bit-for-bit agreement matters.
- **NRSur7dq4 extras.** The polarisation rotation by `longAscNodes` isn't exposed; it is zero in the default `ChooseTDWaveform` call. NRSur7dq2 isn't ported, but the data layer is versioned to allow it. The HDF5 reader only handles the subset that the surrogate files use.
- **Modes and XPNR.** XP has no modes in LAL, so its J-frame modes are checked only against its own polarizations. A co-precessing mode array that leaves out some ℓ drops those J-frame modes, whereas LAL needs `ModeArrayJframe`. The `precession_version` option doesn't apply to XPNR.
- **Conversions.** Agreement with LAL in the time domain is limited to ~1e-12 by LAL's high-pass filter amplifying the last-bit differences between `rustfft` and FFTW.
- **MSA speed.** The MSA angles are only 1.4–1.8× faster than LAL, because the ported elliptic functions dominate.
- **Python bindings.** They are built and tested only on macOS arm64, and not published on PyPI. Wheels for Linux and Windows would need a CI matrix. With `strict=True`, a batch is computed in full before its first error is raised.
- **IMRPhenomXO4a.** The parity fix also applies to LAL's IMRPhenomXO4a, which is not ported.

19 Reproducing the results

```
# environment (conda-forge)
mamba create -n waveformrs -c conda-forge python=3.12 python-lalsimulation numpy scipy matplotlib
conda activate waveformrs

# unit and LALSimulation reference tests
cargo test --release

# regenerate reference data (single-threaded: see section 16)
OMP_NUM_THREADS=1 python scripts/generate_reference_data.py --suites imrphenomxphm_spintaylor

# random draws, timings, figures and this report
OMP_NUM_THREADS=1 python scripts/validate_against_lalsimulation.py \
    --approximants TaylorF2 IMRPhenomD IMRPhenomXAS IMRPhenomXHM IMRPhenomXP IMRPhenomXP_NNLO \
    IMRPhenomXP_MSA IMRPhenomXPHM IMRPhenomXPHM_NNLO IMRPhenomXPHM_MSA --n-draws 200 --outdir report/data
OMP_NUM_THREADS=1 python report/lal_benchmark.py
python report/make_figures.py
python report/build_report.py

# everything at once, on an idle machine
bash report/measure_all.sh

# Linux containers (see docs/linux.md)
bash docker/run.sh arm64 test

# NRSur7dq4 (data in lal_data/, from zenodo.org/records/14999310)
OMP_NUM_THREADS=1 LAL_DATA_PATH=lal_data python scripts/validate_nrsur7dq4.py --n-draws 200 --outdir report/data
OMP_NUM_THREADS=1 python report/nrsur_benchmark.py

# many waveforms in sequence on one grid
OMP_NUM_THREADS=1 python report/serial_benchmark.py --n-draws 300

# the OpenMP race in LALSimulation
python scripts/reproduce_lal_openmp_race.py

# IMRPhenomXPNR with the parity fix: lalsimulation 6.2.0 + lalsuite !2593
CC=/path/to/conda-forge/clang-19 bash scripts/build_patched_lalsimulation.sh ../lalsuite build/lal-2593
OMP_NUM_THREADS=1 python scripts/validate_against_lalsimulation.py --approximants IMRPhenomXPNR_parityfix \
    --lalsimulation-library build/lal-2593/liblalsimulation.37.dylib --n-draws 200 --outdir report/data

# Python bindings
cd python && maturin develop --release && python -m pytest
OMP_NUM_THREADS=1 python benchmarks/benchmark.py --json ../report/data/python_timings.json
```

20 Commit history

The commits on `xphm-spintaylor-validation`, oldest first, each one a logical step. Commits that only add or regenerate this report are not listed. Nothing has been pushed.

COMMIT	TYPE	SUMMARY
f940559	TEST	add lalsimulation reference data and comparison helpers
62ff04a	BUG	fix PN coefficients; add reference frequency to TaylorF2
7cfa480	BUG	make IMRPhenomD agree with lalsimulation
2e1f9f6	MAINT	add LAL physical constants used by IMRPhenomX
5f426f9	PERF	faster TaylorF2 and IMRPhenomD evaluation, drop LAPACK
d44fb0f	PERF	Estrin evaluation of the inspiral PN polynomials
4aa95fb	TEST	random-draw validation harness against lalsimulation
8ec8399	FEAT	port IMRPhenomXAS from LALSimulation
60c53bb	TEST	parse reference data with exact float round trip
2b6c7e9	TEST	compare IMRPhenomXAS against lalsimulation
be10f20	EXAMPLE	add IMRPhenomXAS timing example
71743c8	TEST	include IMRPhenomXAS in validation harness and benchmark
ebea27e	FEAT	port LAL/GSL adaptive RKF45 integrator with Hermite resampling
ff2e5fe	FEAT	add SpinTaylorT4 precessing orbit evolution

COMMIT	TYPE	SUMMARY
3dbb470	TEST	compare SpinTaylorT4 evolution with lalsimulation
f1f53c5	MAINT	add SpinTaylorT4 timing example
db38b62	MAINT	mirror LAL's fused multiply-adds in the spline and final-state fits
9502ee7	FEAT	port IMRPhenomXP with SpinTaylor (320) and NNLO (102) Euler angles
a139f43	TEST	compare IMRPhenomXP against lalsimulation
20189b4	EXAMPLE	add IMRPhenomXP timing example
a5838ac	TEST	include IMRPhenomXP in the random-draw validation harness
7ae6228	DOC	add figure script for the technical report
0c02870	DOC	write up OpenMP data race in lalsimulation IMRPhenomXP MSA/PNR angles
9043fd7	DOC	add PDF copy of the lalsimulation OpenMP race writeup
b11fa66	MAINT	make the IMRPhenomXAS coefficients bit-exact with LALSimulation
e75a0cc	FEAT	port IMRPhenomXHM from LALSimulation
e744ebc	TEST	compare IMRPhenomXHM polarizations against lalsimulation
e84a4b2	MAINT	evaluate the IMRPhenomX (2,2) phase at each frequency as LALSimulation does
3b13a50	PERF	evaluate IMRPhenomXHM modes with LAL's powers of Mf and sin_cos
b800f11	TEST	compare each IMRPhenomXHM mode against lalsimulation
36f9a1d	EXAMPLE	add IMRPhenomXHM timing example
9fa011d	TEST	include IMRPhenomXHM in the random-draw validation harness
4693b55	MAINT	fix clippy lints in the PN, IMRPhenomD and example code
3719da8	PERF	enable fat LTO in the release profile
106939d	FEAT	port the IMRPhenomXP MSA Euler angles (PrecVersion 223)
e265e62	TEST	compare IMRPhenomXP MSA angles and the SpinTaylor fallback with lalsimulation
2812650	TEST	include IMRPhenomXP MSA angles in the random-draw validation
ead6203	PERF	skip the spin-magnitude cubic for degenerate MSA roots
3649640	EXAMPLE	time IMRPhenomXP with the MSA angles
0744a5a	TEST	benchmark every model on a common grid against lalsimulation
98c4b90	TEST	measure the accuracy of the original implementation against lalsimulation
80a0a52	MAINT	mirror LAL's contractions in the NNLO angles and the XP (2,2) mode
235259f	FEAT	port IMRPhenomXPHM with SpinTaylor (320) and NNLO (102) angles
9aa922d	TEST	compare IMRPhenomXPHM against lalsimulation
521fa0f	MAINT	use LAL's powers of Mf for the (2,2) mode of IMRPhenomXP with MSA angles
3a3d325	FEAT	MSA angles and the SpinTaylor-to-MSA fallback in IMRPhenomXPHM
45eb21c	TEST	compare IMRPhenomXPHM MSA angles and the SpinTaylor fallback with lalsimulation
6f1e476	PERF	shorten the SpinTaylor gamma integration for IMRPhenomXPHM
a2d5dfa	TEST	avoid a redundant match guard in the IMRPhenomXPHM test helper
be1618c	EXAMPLE	add IMRPhenomXPHM timing example
f9d386d	TEST	include IMRPhenomXPHM in the random-draw validation harness
64b2e00	EXAMPLE	time IMRPhenomXPHM with the MSA angles
2456a3a	STY	rustfmt the lalsimulation test imports
1365c59	STY	run rustfmt
d2d062e	TEST	add IMRPhenomXPHM to the benchmark
bd90e0b	DOC	the IMRPhenomXPHM loops in lalsimulation are not parallelised
8c09959	TEST	time IMRPhenomXHM/XPHM on uniform grids against LAL multibanding

COMMIT	TYPE	SUMMARY
2fe7cc4	MAINT	ignore the local LAL waveform data directory
c799001	FEAT	FrequencyGrid for evaluating many waveforms on the same frequencies
880982f	PERF	grid evaluation of TaylorF2, IMRPhenomD and IMRPhenomXAS
a962d1b	PERF	grid evaluation of IMRPhenomXHM
d471fda	TEST	check grid evaluation against per-frequency evaluation; serial benchmarks
77830b0	TEST	time the grid paths in the serial benchmark
8b0cce0	FEAT	add a pure-Rust reader for the HDF5 subset of the surrogate data files
af75db7	FEAT	port the precessing NR surrogate NRSur7dq4
1383d17	TEST	compare NRSur7dq4 and the HDF5 reader with lalsimulation and h5py
4b0698d	PERF	read integer datasets without per-element copies
5e9afec	PERF	evaluate NRSur7dq4 about 5x faster than lalsimulation
bf0d141		FIX: evaluate the spherical harmonics with LAL's sin/cos calls
6d3a15c	TEST	compare NRSur7dq4 with SimInspiralChooseTDWaveform over random draws
ec47fc1	PERF	cache the inertial modes in the workspace and resample faster
a05f7e6	EXAMPLE	time NRSur7dq4 against lalsimulation
e09d12b	TEST	cover NRSur7dq4 mode arrays and unlimited extrapolation
98dafe1	DOC	document NRSur7dq4 accuracy, caching and usage
29da81a	TEST	include NRSur7dq4 in the serial benchmark
914b8f0	TEST	benchmark NRSur7dq4 against lalsimulation; add it to the report figures
68842d4	TEST	machine-readable NRSur7dq4 timings for the report
0dbbf9d	DOC	report sections on NRSur7dq4 and on evaluating many waveforms in sequence
e6ab46c	PERF	grid evaluation of IMRPhenomXP and IMRPhenomXPHM
1ee15c5	TEST	IMRPhenomXP and IMRPhenomXPHM grid paths against the per-frequency paths
0159fff	PERF	keep the twisting-up arrays of the grid path in registers
cf3e4f8	PERF	evaluate the MSA angles of the grid paths in passes over blocks
68480d5	PERF	NNLO angles of the grid paths from the powers stored in the grid
73fcebb	PERF	convert the two terms of the (3,2) ringdown separately on the grid
20f3216	TEST	the batched MSA angles give phi_z and zeta bit for bit
eb091c9	TEST	time IMRPhenomXP and IMRPhenomXPHM grid paths in grid_benchmark
049edc0	TEST	alternate the order of the two paths in grid_benchmark
c3fabda	PERF	vectorised merger-ringdown SpinTaylor angles on the grid
3a68e3c	PERF	exp of the equally spaced gamma integration points by powers
56bcf4d	PERF	combine the spherical harmonics of the twisting up per |k|
22a3254	DOC	describe the angle prescriptions of the precessing grid paths
571af30	PERF	stagger the powers stored in FrequencyGrid
de4d46d	STY	simplify the filling of the co-precessing modes on the grid
163cc40	STY	evaluate the MSA angles of IMRPhenomXP on the grid directly
a485183	TEST	time the IMRPhenomXP/XPHM grid paths in the serial benchmark
935f07a	DOC	describe the precessing grid paths and the grid memory layout in the report
6b6d3c7	DOC	note that the grid paths agree with LAL to rounding rather than bit for bit
29eb108	DOC	report fresh-call NRSur7dq4 timings; serial-use notes in the report
089774a	DOC	leave report-regeneration commits out of the report's commit table
e7da9d0	FEAT	phi_c is the orbital reference phase in every Waveform method

COMMIT	TYPE	SUMMARY
5accb7f	FEAT	common model interface with Binary, model traits, Approximant and Error
0507770	FEAT	frequency-domain modes in the conventions of SimInspiralChooseFDModes
ca35f3b	FEAT	switch off the automatic changes of the precession version
6543a39	FEAT	batched evaluation on several threads
573a4ca	TEST	compare long, low-mass waveforms with LALSimulation
3df86d9	PERF	keep only v and LNhat of long SpinTaylor integrations
d751d90	TEST	time every model on long grids against LALSimulation
a157692	TEST	the compact SpinTaylor runs give LAL's PN arrays
4e7b60f	FEAT	LAL's waveform conditioning: duration bounds, tapers, high-pass filter, GPS epochs
09896e8	FEAT	real FFTs with LAL's conventions behind a default `fft` feature
0e1bbc8	FEAT	SimInspiralTD and SimInspiralFD for every approximant of the crate
65074f6	TEST	compare the conversions with SimInspiralTD and SimInspiralFD of LAL 6.2.0
b231157	TEST	timing example for the conversions against LAL
0976c62	PERF	no software fma on x86-64 without the fma target feature
0eb644e	TEST	tolerances for builds the crate does not mirror; FMA choice per platform
0d9172a		BUILD: Linux containers (arm64 native, amd64 emulated) with conda-forge lalsimulation
daa366d		FIX: call glibc's cbrt on linux-gnu, as lalsimulation does
b326cce	TEST	cross-build tolerances for the modes tests
6a7b50b	TEST	Linux validation against the containers' lalsimulation
40e9443	TEST	cross-build tolerances for the conversion and long-waveform tests
477687c	PERF	inline port of glibc's cbrt on linux-gnu
4520bfd	TEST	long SpinTaylor angles and long-waveform floors on emulated amd64
c04485d	DOC	Linux page: containers, test matrix, FMA and cbrt decisions, accuracy, timings
2ed5137		FIX: compile the conversion examples only with the fft feature
0869ef7	FEAT	IMRPhenomXHM release 122019 (original higher-mode amplitudes)
6bec88c	TEST	compare IMRPhenomXHM release 122019 with LALSimulation
d341a3c	FEAT	IMRPhenomXPNR, the phenomenological precessing NR-tuned model
34669f4	FEAT	IMRPhenomXPNR in the common interface, with its J-frame modes
3c919b5	TEST	compare IMRPhenomXPNR with LALSimulation
88666d0		FIX: the C library's cbrt in the PNR angles
b70b798	TEST	cross-build tolerances for IMRPhenomXPNR and IMRPhenomXHM 122019
f8aa52a	PERF	search the intervals of the PNR splines from the previous one
0b7f7d3	TEST	random-draw validation and timing of IMRPhenomXPNR
582ea68	TEST	find the NRSur7dq4 data in lal_data/ in the batch and model tests too
4baea0a	TEST	machine-readable conversion and batch timings; one script for all report measurements
0e360eb	DOC	report sections on long waveforms, IMRPhenomXPNR, the common interface and other platforms
86565e9	DOC	memory in the measure-all script; tidy the conversion and Linux timing tables
5b7dd29	FEAT	Python bindings of the shared interface, with batching
4d117f1	TEST	Python tests against lalsimulation, of batches, errors and the GIL
7fe474c	TEST	benchmark of the Python bindings against lalsimulation's Python API
04a5575		FIX: IMRPhenomXHM returns an error instead of panicking on a negative first frequency
426f840		BUILD: keep the reference data, report and scripts out of the package
965f8e2	TEST	reference data from a patched lalsimulation through ctypes

COMMIT	TYPE	SUMMARY
ce8fa02		FIX: IMRPhenomXPNR with the parity fix of lalsuite !2593
389ebe7	FEAT	a grid path for IMRPhenomXPNR
7762500	DOC	the IMRPhenomXPNR parity fix and grid path in the report template
aa1f45c	TEST	fall back to tests/data for suites missing from WAVEFORMRS_REFERENCE_DATA
4fe9857	FEAT	the IMRPhenomXPNR parity-fix option in the Python bindings